

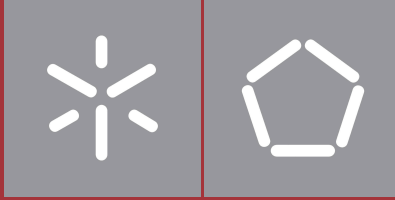


Universidade do Minho
Escola de Engenharia

Luís Paulo Peixoto dos Santos

From Light Paths to Feynman Paths

Provas de Agregação em Informática
Habilitation in Computer Science
Lição / Seminar



Universidade do Minho
Escola de Engenharia

Luís Paulo Peixoto dos Santos

From Light Paths to Feynman Paths

Provas de Agregação em Informática
Habilitation in Computer Science

Lição / Seminar



Direitos de Autor e Condições de Utilização do Trabalho por Terceiros

Este é um trabalho académico que pode ser utilizado por terceiros desde que respeitadas as regras e boas práticas internacionalmente aceites, no que concerne aos direitos de autor e direitos conexos.

Assim, o presente trabalho pode ser utilizado nos termos previstos na licença abaixo indicada.

Caso o utilizador necessite de permissão para poder fazer um uso do trabalho em condições não previstas no licenciamento indicado, deverá contactar o autor, através do RepositóriUM da Universidade do Minho.

Licença concedida aos utilizadores deste trabalho:



CC BY-SA

<https://creativecommons.org/licenses/by-sa/4.0/>

Declaração de Integridade

Declaro ter atuado com integridade na elaboração do presente trabalho académico e confirmo que não recorri à prática de plágio nem a qualquer forma de utilização indevida ou falsificação de informações ou resultados em nenhuma das etapas conducente à sua elaboração.

Mais declaro que conheço e que respeitei o Código de Conduta Ética da Universidade do Minho.

Universidade do Minho, Braga, setembro 2025

Luís Paulo Peixoto dos Santos

Resumo

Este texto constitui o relatório submetido à Universidade do Minho para preenchimento parcial dos requisitos para realização de Provas de Agregação no ramo do conhecimento de Informática nos termos do número 2 do artigo 8º do DL 239/2007, de 19 de Junho, cuja alínea c) menciona “... *um seminário ou lição sobre um tema dentro do âmbito do ramo do conhecimento ou especialidade em que são prestadas as provas...*”.

Esta lição aborda o problema da simulação clássica de circuitos quânticos. A hipótese colocada é de que métodos de Monte Carlo habitualmente utilizados no contexto da Iluminação Global, podem ser utilizados para nestes simuladores. O algoritmo em questão é designado por *BiDirectional Path Tracing*. Esta hipótese é testada usando um simulador baseado na soma de caminhos de Feynman, indicando que este algoritmo localiza com grande probabilidade os caminhos com amplitude diferente de zero, mesmo quando este são uma fracção mínima do total de caminhos.

Palavras-chave Seminário, Agregação, Integração de Monte Carlo, Simulação Clássica de Circuitos Quânticos

Abstract

This document intends to partially fulfill the requirements of University of Minho to grant the academic title of Habilitation in Computer Science.

This lecture addresses classical simulation of quantum circuits. The hypothesis is that Monte Carlo methods, specifically *BiDirectional Path Tracing*, used in the context of Global Illumination, can be applied to these simulations. This hypothesis is tested using a Monte Carlo simulator based on the Feynman sum of paths formulation of quantum mechanics. Results show that the proposed algorithm can identify, with large probability, paths with non-zero amplitude, even when these are a small fraction of all paths connecting the initial to the final state.

Keywords Seminar, Habilitation, Monte Carlo Path Integration, Classical Simulation of Quantum Circuits

Contents

Preface	xiii
1 Introduction	1
2 Monte Carlo Methods' Fundamentals	3
2.1 Monte Carlo integration	4
2.2 Importance Sampling	5
2.3 Multiple Importance Sampling	6
3 Light Transport	9
3.1 The Light Transport Problem and Equation	9
3.2 Path Integration	12
3.3 BiDirectional Path Tracing	14
4 Quantum Circuits and Feynman Sum of Paths	16
4.1 The Hamiltonian Formalism	16
4.2 Quantum Circuits	17
4.3 Path Integral Formalism	19
4.4 Discussion	23

5	Path Integration Monte Carlo	24
5.1	Forward Uniform Sampling Path Tracing	25
5.2	Forward Importance Sampling Path Tracing	26
5.2.1	Importance Sampling the Next State	28
5.3	Bidirectional Path Tracing	31
5.4	Multiple Importance Sampling Bidirectional Path Tracing	33
5.5	Practical Considerations	36
6	Experimental Testing	38
6.1	Methodology	38
6.1.1	Circuits	39
6.2	Results	42
6.2.1	Convergence Comparison	42
6.2.2	Scalability with Circuit Dimension	47
6.2.3	Very Large Hidden Shift Circuits	49
6.3	Discussion	49
7	Concluding Remarks	51
A	Circuit Layer Level Methods	58
A.1	Preliminaries	58
A.2	Layer Amplitude Evaluation	59
A.3	Layer Sampling	59
A.4	Layer Transition Probability	60

List of Figures

1	Document Structure	2
2	Simple setup for light transport simulation	10
3	Hemisphere sampling	11
4	Path tracing	12
5	Path tracing - low probability path	14
6	BiDirectional Path Tracing: forward and backward paths	14
7	BiDirectional Path Tracing: connecting vertices on the forward and backward paths	15
8	Different possibilities for sampling $\{0, q_o, p_0, \dot{L}\}$ (besides figure 7)	15
9	Unitaries are composed in time via multiplication $\hat{U} = \hat{U}_2 * \hat{U}_1 * \hat{U}_0$	18
10	Unitaries are composed in space via the tensor product: $\hat{U}_0 = \hat{U}_{0,(0,1)} \otimes \mathbb{I}_{(2)}$; $\hat{U}_1 = \hat{U}_{1,(0)} \otimes \hat{U}_{1,(1)} \otimes \hat{U}_{1,(2)}$; $\hat{U}_2 = \mathbb{I}_{(0)} \otimes \hat{U}_{2,1} \otimes \mathbb{I}_{(2)}$	18
11	3 qubits, 4 layers quantum circuit and respective paths	21
12	Analogy of information propagation on a quantum circuit with light transport.	25
13	Forward importance sampling path tracing: up to layer $L - 1$ (green arrows) the path is guaranteed to have both non-zero amplitude and probability. The deterministic connection to the final state (orange dashed arrow) has probability 1, but its amplitude $\langle i \hat{U}_{L-1} t_{L-1} \rangle$ can be zero.	27

14	A typical layer to sample a classical state $ t_{l+1}\rangle$ given $ t_l\rangle$	30
15	Forward and backward importance sampled path segments for $L = 4$	31
16	Paths generated by connecting forward and backward path segments. The dashed dark orange arrow represents the deterministic connection. Non-used path segments are dimmed to grey.	32
17	$L + 1$ different manners the same path, originally deterministically connected across L as depicted by the blue arrow (see figure 16b), could have been generated by shifting the deterministic connection (dark orange dashed arrow).	33
18	Variance as a function of the number of Monte Carlo samples	43
19	Variance as a function of time	44
20	Efficiency as a function of the number of samples	46
21	Number of non-zero amplitude paths as a function of time	47
22	Scalability with circuit dimension: RSQ8 , RSQ12 and RSQ16	48

List of Algorithms

1	Canonical Feynman sum of paths	22
2	Forward Uniform Sampling Path Tracing	26
3	Forward Importance Sampling Path Tracing	28
4	Bidirectional Path Tracing	32
5	Multiple Importance Sampling Bidirectional Path Tracing	35
6	Sample Layer	60

List of Tables

1	Output states and respective probability for the example presented in figure 14.	31
2	Circuits used in experimental testing (n - qubits, L - layers, #paths - total number of paths, #NZ – number of non-zero paths). Evaluating the number of different non-zero paths requires the execution of some deterministic algorithm that iterates through all possible paths that are not pruned. This is a computationally very demanding task; therefore, values are not available (n.a.) for larger circuits.	41
3	Square Random circuits used to evaluate scalability (n - qubits, L - layers, #paths - total number of paths, #NZ – number of non-zero paths).	41
4	BDPT simulation of very large Hidden Shift Circuits	50

Acronyms

BDPT BiDirectional Path Tracing.

BRDF BiDirectional Reflection Distribution Function.

GI Global Illumination.

HSC Hidden Shift Circuits.

MC Monte Carlo.

MIS Multiple Importance Sampling.

PIMC Path Integral Monte Carlo.

QC Quantum Computing.

RSQ Random Square Circuits.

Preface

Physically based **Global Illumination (GI)** is the art and science of estimating a solution for the light transport integral within a time and space limited budget of computational resources. Around 1948 Richard Feynman [1] taught us that light can be understood as propagating simultaneously through all possible paths between the light source and the light sensor (e.g., the observer's eye). Evaluating some metric proportional to the light's power, as measured by the sensor, implies a weighted sum over the infinitely many possible light paths, a process referred to as path integration.

Modern days **GI** algorithms resort to **Monte Carlo (MC)** numerical quadratures to estimate the light transport integral. Paths are built and weighted stochastically, sampling from wisely selected probability distributions, a process known as path tracing or, more generally, **MC path integration**. These algorithms are often evaluated by how fast they converge to the integral expected value, i.e., how variance changes as a function of the number of samples. The human visual system perceives variance as noise: research results in **GI** typically imply quantitative and qualitative comparisons of noisy images rendered by competing algorithms.

This has been the territory of my research since 2003. It was a good match for both my intellectual interests and expertise. The physically based light transport equation confines the research space to the boundaries imposed by mathematical formalisms, allowing for rigorous verifications of the soundness of potential results. Simultaneously, the subjectivity associated with the human visual system allows for challenging degrees of freedom in cognitive and perception related tasks. Additionally, parallel processing, the core area of my PhD thesis, is a major resource exploited by practical **GI** frameworks. Whereas naïve Monte Carlo is embarrassingly parallel, more aggressive **MC** algorithms require efficient sharing of runtime computed knowledge, posing challenges that can only be addressed by leveraging the expertise gathered as a parallel processing researcher.

In 2018 I shifted my research area to **Quantum Computing (QC)**, a paradigm that has been attracting

growing research efforts and funding over the last decade. **QC** abides by a set of rules that are very different from the rules governing classical computation. These include concepts such as superposition of states, entanglement, interference, no-cloning, and no-locality, which are non-intuitive and feel like alien formalisms to the non-initiated practitioner. Additionally, **QC** practice is still very much dependent on knowledge of quantum mechanics. This constitutes an additional barrier to adoption for those with a computer science background but lacking strong foundations in physics.

The question I posed myself when embracing this challenging research field was:

Can a bridge (a path) be established connecting efficient Monte Carlo light path integration algorithms with innovative solutions to pending quantum computing problems?

I quickly found that the answer is affirmative. In fact, quantum computation is often expressed using Feynman's sum-over-histories formulation of quantum mechanics [2, 1]. A history is a sequence, or a path, of classical states starting on a given initial state and terminating on some final state. The probability amplitude of a transition from the initial to the final classical states is given by the integral, over every conceivable path, of a function of the path. Classical simulation of quantum circuits can thus be expressed in terms of Feynman's sum of histories (paths). **Path integration** emerges as a fundamental mathematical tool, enabling such simulations in this formulation. This is known as Feynman's approach to simulation, an alternative to Schrodinger's approach. Applying **MC** to Feynman's formulation of quantum mechanics is known as **Path Integral Monte Carlo (PIMC)**, and constitutes an obvious bridge between **GI** and **QC**.

This seminar presents my earlier efforts to apply a well-known (within the **GI** community) Monte Carlo light path integration algorithm to the summation of Feynman paths, allowing for the efficient classical simulation of (some) quantum circuits.

Chapter 1

Introduction

It is a truth very certain that when it is not in our power to determine what is true we ought to follow what is most probable.

Discourse on Method

Descartes

This seminar presents the results of applying a well-known **Monte Carlo (MC)** light path integration algorithm, referred to as **BiDirectional Path Tracing (BDPT)** [3], to the problem of classically simulating quantum circuits.

MC integration is currently the most important mathematical formalism used by the **GI** community to estimate light transport from light sources to light sensors. Its relevance within the area led to the proposal of a wide variety of **MC** integration algorithms in a quest to improve the convergence rate, even when facing challenging scenarios. A large family of such **MC** methods applies to path integral formulations of light transport [4]; a weighted sum of the contributions of a large number of light paths from the light source to the sensor estimates the quantity of interest. **BDPT** is one such algorithm tailored to handle problems where light paths connecting the source to the sensor have a low probability of being found.

MC methods are widely used to estimate properties of complex quantum systems [5]. This approach is referred to as Quantum Monte Carlo, comprising a vast range of algorithms based on the **MC** method to evaluate multidimensional integrals used to describe quantum systems. Resorting to Feynman's sum-over-histories formulation of quantum mechanics [2, 6] enables the utilization of **Path Integral Monte Carlo (PIMC)** algorithms.

A relationship between **GI** and quantum simulation becomes evident. Path integrals are objects of utmost

relevance in both areas of knowledge. Methods and results familiar to any of these communities must provide valuable insights to the other community.

The goals of this lecture can be summarized as:

- demonstrate how **BDPT** can be applied to classically approximately simulating quantum circuits;
- discuss insights and challenges identified throughout the process.

This lecture is structured as follows (figure 1). Chapter 2 reviews some fundamental concepts of Monte Carlo integration and path integrals. The following chapter presents the light transport equation and the canonical **MC** path tracing algorithm. Chapter 4 discusses Feynman's classical simulation of quantum circuits. Chapter 5 presents an analogy from light path space to Feynman's quantum histories space and describes the global illumination-inspired bidirectional path integration algorithms applied to the classical simulation of quantum circuits. Chapter 6 experimentally tests, for a range of quantum circuits, whether the proposed bidirectional algorithm improves convergence relative to a forward-only algorithm. The document closes with some concluding remarks included in chapter 7.

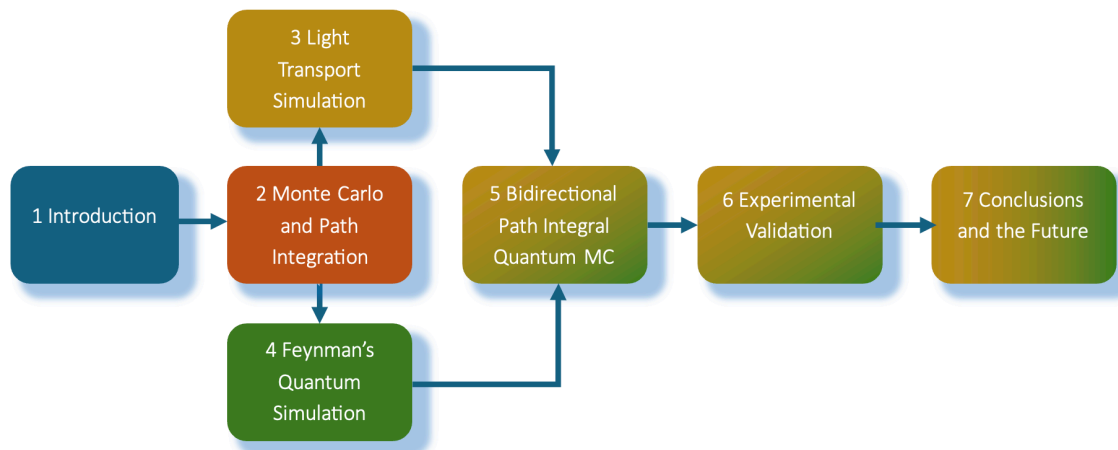


Figure 1: Document Structure

Chapter 2

Monte Carlo Methods' Fundamentals

The first thoughts and attempts I made to practice [the Monte Carlo Method] were suggested by a question which occurred to me in 1946 as I was convalescing from an illness and playing solitaires. The question was what are the chances that a Canfield solitaire laid out with 52 cards will come out successfully? After spending a lot of time trying to estimate them by pure combinatorial calculations, I wondered whether a more practical method than 'abstract thinking' might not be to lay it out say one hundred times and simply observe and count the number of successful plays.

Stanislaw Ulam

It is not straightforward to arrive at an exact definition of what constitutes a **Monte Carlo (MC)** method. In general, it can be said that **MC** methods require using random numbers to estimate some quantity of interest. The associated calculations exhibit the structure of a stochastic process, i.e., a sequence of states whose evolution is determined by random events [7].

It is essential to keep in mind that although the Monte Carlo computation is a stochastic process, the quantity being estimated does not have to be, originally, a stochastic variable. Stochasticity is deliberately introduced in the calculation due to a wide range of alternative reasons, such as uncertainty or ignorance about the process that causes the quantity being estimated. Or due to limited time and space computing resources. Or because, in fact, the underlying process is understood as being stochastic (which does not preclude the possibility that this understanding is due to ignorance...).

2.1 Monte Carlo integration

This lecture focuses on estimating integrals (or sums) over high-dimensional spaces. **MC** excels in such spaces since it does not imply exponential growth of the time complexity with the number of dimensions; deterministic approaches have been shown to require a number of samples exponential on the number of dimensions for constant error [4], which is referred to as the curse of dimensionality. The goal is to evaluate the estimator $\hat{I}$ of the d -dimensional integral I (with d eventually infinite):

$$\hat{I} \approx I = \int_{V^{(d)}} \dots \int_{V^{(1)}} f(x^{(1)}, \dots, x^{(d)}) dx^{(1)} \dots dx^{(d)}$$

where $V^{(t)}$ is the domain of integration of dimension t and $V = \bigotimes_{t=1}^d V^{(t)}$ is the d -dimensional integration space.

If a point $x = (x^{(1)}, \dots, x^{(d)})$ is stochastically selected in V , with probability $p(x)$, and a sample $f(x)$ is evaluated, then $\hat{I}_0$ is a primary estimator of I :

$$\hat{I}_0 = \frac{f(x)}{p(x)} \approx I$$

For N samples $f(x_j)$, $j = 0 \dots N - 1$, each selected with probability $p(x_j)$, a **MC** estimator is the arithmetic mean of the respective primary estimators:

$$\hat{I}_N = \frac{1}{N} \sum_{j=0}^{N-1} \hat{I}_0(x_j) = \frac{1}{N} \sum_{j=0}^{N-1} \frac{f(x_j)}{p(x_j)} \approx I \quad (2.1)$$

Since the expected value of a stochastic function is $E(f(X)) = \int f(x) p(x) dx$ then

$$\begin{aligned} E(\hat{I}_N) &= E\left(\frac{1}{N} \sum_{j=0}^{N-1} \frac{f(x_j)}{p(x_j)}\right) \\ &= \frac{1}{N} \sum_{j=0}^{N-1} E\left(\frac{f(x_j)}{p(x_j)}\right) \\ &= \frac{1}{N} \sum_{j=0}^{N-1} \int \frac{f(x) p(x)}{p(x)} dx \\ &= \int_V f(x) dx. \end{aligned}$$

The variance is the second central moment of a random variable. Variance is especially important because its square root is a measure of the dispersion of the random variable, $\sigma(X) = \sqrt{\text{Var}(X)}$, which is referred to as the standard deviation and sometimes the standard error.

The variance of a random function F is defined to be $Var(F) = E[(F - E(F))^2]$. The following alternative formulation and properties are known to hold:

- $Var(F) = E(F^2) - E^2(F)$
- $Var(aF) = a^2 Var(F)$
- $Var(\sum_i F_i) = \sum_i Var(F_i)$, if the F_i are independent random variables.

The variance of the integral estimator is, therefore, given by [4]

$$\begin{aligned}
 Var(\hat{I}_N(X)) &= \frac{1}{N^2} Var\left(\sum_{j=0}^{N-1} \frac{f(x_j)}{p(x_j)}\right) \\
 &= \frac{1}{N^2} \sum_{j=0}^{N-1} Var\left(\frac{f(x_j)}{p(x_j)}\right) \\
 &= \frac{1}{N} Var\left(\frac{f(X)}{p(X)}\right) \\
 &= \frac{1}{N} \left[E\left(\left[\frac{f(X)}{p(X)}\right]^2\right) - E^2\left(\frac{f(X)}{p(X)}\right) \right] \\
 &= \frac{1}{N} \left[\int \left(\frac{f^2(X)}{p(X)}\right) - I^2 \right].
 \end{aligned} \tag{2.2}$$

Equation 2.2 illustrates two important properties of our estimator:

- the variance decreases linearly with the number of samples N . Therefore, the standard error, σ , falls with $\sqrt{N}$ (e.g., four times more samples are required to halve the error);
- the probability, $p(X)$, with which samples are selected significantly impacts variance. In particular, if $p(X)$ is small in regions of the integration domain where $f(X)$ is large, then the variance will increase.

2.2 Importance Sampling

If there is no *a priori* knowledge about the shape of $f(x)$, samples are drawn from a uniform probability distribution, a constant whose value is equal to the volume of the integration domain V , that is, $p(x) = \frac{1}{\|V\|}$. The estimator is given by

$$\int_V f(x) dx \approx \hat{I}_{IS} = \frac{1}{N} \sum_{j=0}^{N-1} \frac{f(x_j)}{p(x_j)} = \frac{\|V\|}{N} \sum_{j=0}^{N-1} f(x_j).$$

But if there is some knowledge of the shape of $f(x)$, then the sampling distribution $p(x)$ can be chosen to have a similar shape, such that $\frac{f(x)}{p(x)} \leq C, C \in \mathbb{R}, \forall x \in V$. By substituting into equation 2.2

$$\begin{aligned} Var(\hat{I}_{IS}(X)) &= \frac{1}{N} \left[\int \frac{f^2(X)}{p(X)} dx - I^2 \right] \\ &\leq \frac{1}{N} \left[C \int f(X) dx - I^2 \right]. \end{aligned} \tag{2.3}$$

Equation 2.3 shows that an appropriate choice of $p(x)$ decreases the variance by decreasing the subtraction's minuend.

In the ideal case $p(x)$ would have the same shape as $f(x)$, resulting in $p(x) = \frac{1}{C_0} f(x)$, where $\frac{1}{C_0}$ is a normalizing constant ($\int_V p(x) dx = \frac{1}{C_0} \int_V f(x) dx = 1 \Leftrightarrow C_0 = \int_V f(x) dx$). The variance, as given by equation 2.2, would be 0, and a single sample would suffice to exactly compute the integral. C_0 is C minimum possible value in equation 2.3. Such a choice of $p(x)$ is not feasible since it requires knowledge of the value of $I = \int_V f(x) dx$, invalidating the whole reasoning.

Importance sampling requires selecting an analytically integrable function $g(x)$ believed to be similar to $f(x)$ and making $p(x) = \frac{1}{C} g(x)$, with $C = \int_V g(x) dx$. In order to efficiently sample from $p(x)$ its cumulative distribution function, $cdf(x)$, must be invertible such that

$$cdf(x) = \int_V^x p(x) dx \text{ and } x = cdf^{-1}(\xi), \xi \in [0 \dots 1]^d$$

with some notation abuse due to the d -dimensional nature of the integration domain.

2.3 Multiple Importance Sampling

Selecting an appropriate sampling distribution is often impossible when the integrand is somehow *difficult*. $f(x)$ can be discontinuous and/or dependent on parameters unknown at design time and/or be a function (e.g., a product) of multiple different functions. Under such conditions, proposing a low-variance **MC** estimator might be possible by combining samples from multiple sampling distributions.

Let $f(x) = \prod_j f_j(x)$. If finding a $p(x) \propto f(x)$ under the conditions mentioned in section 2.2 is unfeasible, then multiple sampling distributions $p_i(x)$ can be used, each proportional to a particular $f_j(x)$

or to the product of some subset of $f_j(x)$. **Multiple Importance Sampling (MIS)** allows combining samples taken from these different distributions in a manner that can be proved to be close to optimum [4].

Given P different sampling distributions $p_i(x), i \in \{0, \dots, P-1\}$, **MIS** requires that:

- $f(x)$ and all $p_i(x)$ can be evaluated $\forall x \in V$;
- a sample $x_j \in V$ can be generated from any of the $p_i()$.

The balance heuristic was proposed by [4] and shown to be close to optimum.

The first step is to stochastically select which $p_i()$ to use, with a probability distribution c over the P sampling distributions, $c = \{c_0, \dots, c_{P-1}\}$, $\sum_0^{P-1} c_i = 1$.

The **MC MIS** estimator is given by

$$\hat{I}_{MIS}(X) = \frac{1}{N} \sum_{j=0}^{N-1} w(x_j) \frac{1}{c_{j,i}} \frac{f(x_j)}{p_i(x_j)} \quad (2.4)$$

The balance heuristic proposes weights w_j that make the sample contribution independent of the probability distribution with which it was drawn, that is,

$$w(x) = \frac{p_i(x)}{\sum_{t=0}^P p_t(x)} \quad (2.5)$$

By substituting 2.5 into equation 2.4 we get the **MIS** estimator

$$\begin{aligned} \hat{I}_{MIS}(X) &= \frac{1}{N} \sum_{j=0}^{N-1} w(x_j) \frac{1}{c_{j,i}} \frac{f(x_j)}{p_i(x_j)} \\ &= \frac{1}{N} \sum_{j=0}^{N-1} \frac{p_i(x_j)}{\sum_{t=0}^{P-1} p_t(x_j)} \frac{1}{c_{j,i}} \frac{f(x_j)}{p_i(x_j)} \\ &= \frac{1}{N} \sum_{j=0}^{N-1} \frac{1}{c_{j,i}} \frac{f(x_j)}{\sum_{t=0}^{P-1} p_t(x_j)} \end{aligned} \quad (2.6)$$

In other words, when finding a sampling distribution $p(x)$ similar to $f(x)$ is not possible, then it might be possible to find multiple $p_i(x)$ that sample $f(x)$ in different ways. For example, if $f(x) = \prod_{i=0}^{P-1} f_i(x)$, then maybe P sampling distributions can be found such that $p_i(x) \propto f_i(x)$.

Equation 2.6 tells us to, for each sample of a total of N samples, start by stochastically selecting the sampling distribution $p_i()$ with probability $c_{j,i}$. Then generate sample $x_j \in V$ using $p_i()$. Compute $f(x_j)$ and the sum of the probabilities of all the manners x_j could have been generated, $\sum_{t=0}^{P-1} p_t(x_j)$. By applying equation 2.6 to all N samples, a robust low variance estimate of the integral of $f(x)$ in V is obtained.

To estimate the variance of the **MIS** estimator let us simplify and assume that the sampling distribution to use at each step is selected with an uniform distribution, i.e., $c_{j,i} = \frac{1}{P}$. Then the variance is given by

$$\begin{aligned}
Var(\hat{I}_{MIS}(X)) &= \frac{P^2}{N^2} Var\left(\sum_{j=0}^{N-1} \frac{f(x_j)}{\sum_{t=0}^{P-1} p_t(x_j)}\right) \\
&= \frac{P^2}{N^2} \sum_{j=0}^{N-1} Var\left(\frac{f(x_j)}{\sum_{t=0}^{P-1} p_t(x_j)}\right) \\
&= \frac{P^2}{N} Var\left(\frac{f(X)}{\sum_{t=0}^{P-1} p_t(X)}\right) \\
&= \frac{P^2}{N} \left[E\left(\left[\frac{f(X)}{\sum_{t=0}^{P-1} p_t(X)}\right]^2\right) - E^2\left(\frac{f(X)}{\sum_{t=0}^{P-1} p_t(X)}\right) \right] \\
&= \frac{P^2}{N} \left[\int \left(\frac{f(X)}{\sum_{t=0}^{P-1} p_t(X)}\right)^2 p(X) dX - I^2 \right].
\end{aligned} \tag{2.7}$$

Recalling that by definition since $p_t(X) \geq 0$, then the sum of several positive numbers is larger than each of them, $\sum_{t=0}^{P-1} p_t(X) \geq p_i(X)$, $\forall i \in \{0, \dots, P-1\}$, therefore the minuend in equation 2.7 is always less than or equal to the minuend in the importance sampling variance estimator $Var(\hat{I}_{IS})$ given by equation 2.3. As long as the reduction in the minuend is larger than or equal to P^2 , then $Var(\hat{I}_{MIS}) \leq Var(\hat{I}_{IS})$.

Chapter 3

Light Transport

Paths are generated by following a random walk through path space, such that the probability density of visiting each path is proportional to the contribution it makes to the ideal image.

Robust Monte Carlo Methods for Light Transport Simulation

(Stanford, 1997)

Eric Veach

Light transport is the process by which light propagates, interacts, and scatters within a scene. It addresses the physical principles determining how light travels, reflects, refracts, absorbs, and emits from surfaces and materials. Light transport is a fundamental concept for computer graphics. Algorithms that render realistic images, simulating the physical laws of light emission, scattering, and interaction, are referred to as light transport algorithms.

This lecture introduces the fundamentals of light transport with the minimum depth deemed necessary to suggest an analogy with the simulation of quantum circuits.

3.1 The Light Transport Problem and Equation

Light transport algorithms estimate a measurement (or a finite number of measurements) of radiance. Radiance is defined as the radiant power per unit area and unit solid angle. A measurement is the response of a sensor. For example, a virtual camera can be thought of as a matrix of sensors, with the pixels' values being proportional to the total radiance incident in each sensor. In the context of this lecture,

we will simplify the terminology and refer to the light transport calculation as the estimate of a pixel value.

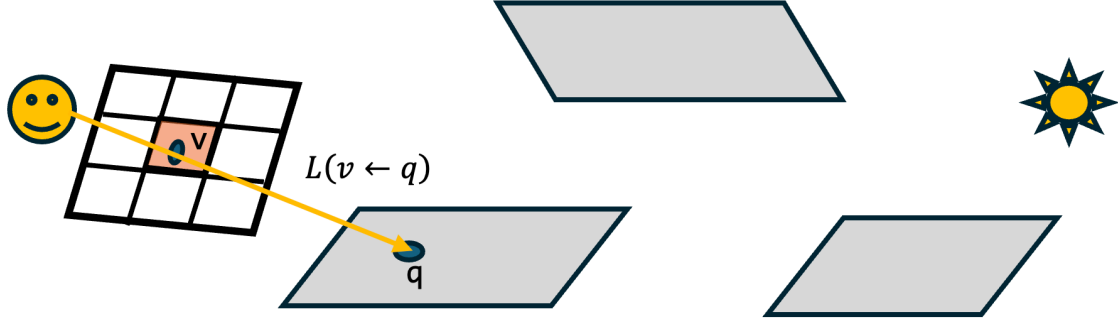


Figure 2: Simple setup for light transport simulation

Figure 2 illustrates a simple setup for light transport simulation. The sensors are on the far left and determine a virtual image plane, discretized into pixels. The scene is composed of a few geometric objects, and a light source sits on the far right. A primary ray is shot from the observer through a stochastically selected point, v , within the pixel's virtual area. The ray intersects the scene's geometry at point $q = RT(o, v)$. The function $RT(o, v)$ models the tracing of a ray with origin at o , the direction given by $v - o$, and returning an intersection point. The rendering, or light transport, problem is to estimate the radiance scattered from q in the direction of v : $L(v \leftarrow q)$. The pixel value will be proportional to the integral of the incoming radiance over the pixel virtual area, which can be estimated using **Monte Carlo (MC)** integration by taking N_{spp} samples, where *spp* stands for *samples per pixel*. The estimate is given by equation 3.1, assuming the pixel virtual area A is equal to 1 and uniformly sampled with $p(v_i) = \frac{1}{A} = 1$:

$$\begin{aligned}
 I_{pixel} &\propto \int_A L(v \leftarrow RT(o, v)) dv \\
 &\approx \frac{1}{N_{spp}} \sum_i \frac{L(v_i \leftarrow RT(o, v_i))}{p(v_i)} \\
 &= \frac{1}{N_{spp}} \sum_i L(v_i \leftarrow RT(o, v_i))
 \end{aligned} \tag{3.1}$$

The rendering equation [8] models the radiance scattered by a point q in a given direction ω_o , $L(\omega_o \leftarrow q)$, as a function of the auto-emitted radiance, $L_e(\omega_o \leftarrow q)$, plus radiance scattered at q , $L_s(\omega_o \leftarrow q)$, due to radiance incident from all directions defined by the hemisphere Ω centered at q :

$$\begin{aligned}
L(\omega_o \leftarrow q) &= L_e(\omega_o \leftarrow q) + L_s(\omega_o \leftarrow q) \\
&= L_e(\omega_o \leftarrow q) + \int_{\Omega} f_r(\omega_o \leftarrow q \leftarrow \omega_i) L(q \leftarrow \omega_i) \cos(\vec{N}_q, \omega_i) d\omega_i
\end{aligned} \tag{3.2}$$

where $\vec{N}_q$ is the normal to the surface at q . $f_r(\omega_o \leftarrow q \leftarrow \omega_i)$ is the **BiDirectional Reflection Distribution Function (BRDF)** characterizing the point material properties, i.e., which fraction of the radiance incident along ω_i is scattered to ω_o .

In the context of this lecture it is assumed that a material either emits or scatters radiance. This rather strong assumption means that only one of the terms of equation 3.2 can be different from zero (both $L_e(\omega_o \leftarrow q) \neq 0$ and $L_s(\omega_o \leftarrow q) \neq 0$ is not allowed).

The scattered radiance can be estimated by **MC** integrating over Ω , sampling N directions ω_i with some probability $p(\omega_i)$:

$$\begin{aligned}
L_s(\omega_o \leftarrow q) &= \int_{\Omega} f_r(\omega_o \leftarrow q \leftarrow \omega_i) \cos(\vec{N}_q, \omega_i) L(q \leftarrow \omega_i) d\omega_i \\
&\approx \frac{1}{N} \sum_i \left[\frac{f_r(\omega_o \leftarrow q \leftarrow \omega_i) \cos(\vec{N}_q, \omega_i)}{p(\omega_i)} * L(q \leftarrow \omega_i) \right]
\end{aligned} \tag{3.3}$$

Figure 3 illustrates the hemisphere sampling process for some representative ω_i . A ray is traced from q along ω_i returning $q' = RT(q, \omega_i)$. Evaluating $L(q \leftarrow \omega_i)$ in equation 3.3 amounts to evaluating $L(-\omega_i \leftarrow RT(q, \omega_i)) = L(-\omega_i \leftarrow q')$. In this lecture context we assume that radiance is propagating in the vacuum, thus $L(q \leftarrow \omega_i) = L(-\omega_i \leftarrow q')$.

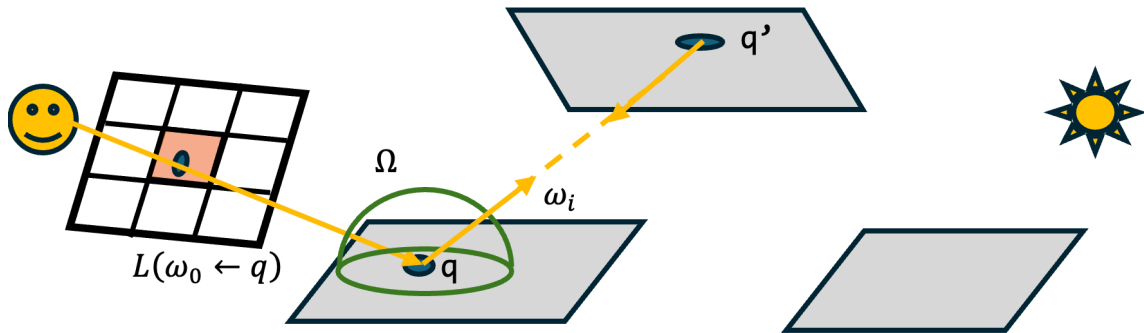


Figure 3: Hemisphere sampling

The light transport equation (3.2) must thus be evaluated at q' to solve $L(\omega_o \leftarrow q)$. This is a recursive equation, with eventually infinite levels of recursion in a closed environment.

In this lecture's context we assume some criterion to stop recursion and further assume that the light source is **explicitly** sampled when recursion terminates. Further, we set $N = 1$ in equation 3.3, meaning that a single incident direction ω_i is sampled to estimate $L_s(\omega_o \leftarrow q)$. Therefore the branching factor of our tree of rays at each object interaction is one.

For each sample v_i in the virtual area of the pixel (Equation 3.1), $L(v_i \leftarrow RT(o, v_i))$ is estimated as a random walk, or a path, from the observer to the light source. Integration is performed over the pixel area by evaluating multiple paths, as determined by N_{spp} and illustrated in figure 4 for $N_{spp} = 2$. The color of the shadow rays illustrates the fact that these directions are selected deterministically rather than stochastically. This algorithm, and variations thereof, is referred to as **path tracing**.

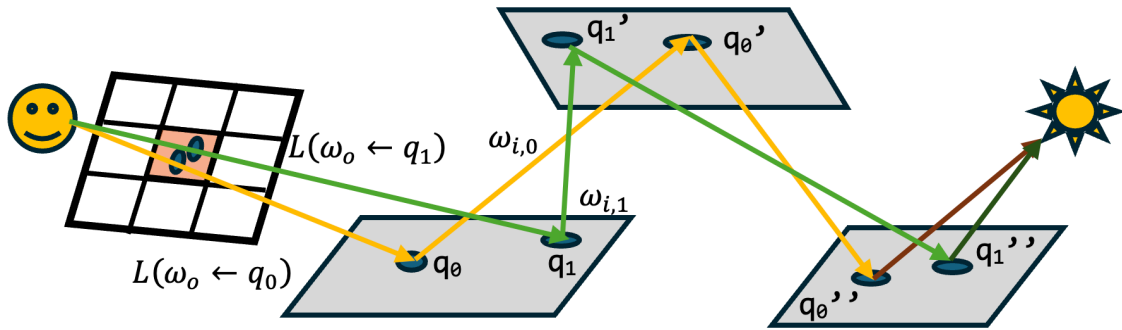


Figure 4: Path tracing

3.2 Path Integration

A path can thus be defined as a stochastically sampled sequence of points from the observer to the light source, with arbitrary length l . A path $\bar{p}$ with length l has $l + 1$ vertices, $\{o, q_0, \dots, q_{l-2}, \dot{L}\}$, with o being the virtual camera, q_0 corresponding to a point visible from the camera and $\dot{L}$ to a point in a light source.

The amplitude of a path $\bar{p}$ is given by $f(\bar{p})$, and the probability of generating a particular path is $p(\bar{p})$. In the very specific context within which light transport paths have been defined, these quantities, for length

l , are given by

$$\begin{aligned}
f(\bar{p}_l) = & f_r(o \leftarrow q_0 \leftarrow q_1) \cos(\vec{N}_{q_0}, q_0 \rightarrow q_1) * \\
& * \prod_{j=1}^{l-3} \left[f_r(q_{j-1} \leftarrow q_j \leftarrow q_{j+1}) \cos(\vec{N}_{q_j}, q_j \rightarrow q_{j+1}) \right] * \\
& f_r(q_{l-3} \leftarrow q_{l-2} \leftarrow \dot{L}) \frac{\cos(\vec{N}_{q_{l-2}}, q_{l-2} \rightarrow \dot{L}) * \cos(\vec{N}_{\dot{L}}, \dot{L} \rightarrow q_{l-2})}{distance^2(q_{l-2}, \dot{L})} L(q_{l-2} \leftarrow \dot{L}) \quad (3.4) \\
p(\bar{p}_l) = & p(o \rightarrow q_0) * \prod_{j=0}^{l-3} p(q_j \rightarrow q_{j+1}) , \text{ conditioned on } p(q_{l-2} \rightarrow \dot{L}) = 1
\end{aligned}$$

Note that the last connection from q_{l-2} to $\dot{L}$ is deterministic rather than stochastically selected. Its probability is, therefore, 1. Furthermore, the integration measure is different, requiring the introduction of the ratio $\frac{\cos(\vec{N}_{\dot{L}}, \dot{L} \rightarrow q_{l-2})}{distance^2(q_{l-2}, \dot{L})}$. The geometric term

$$G(p, q) = \frac{\cos(\vec{N}_p, p \rightarrow q) * \cos(\vec{N}_q, q \rightarrow p)}{distance^2(p, q)}$$

is introduced in the light transport equation when sampling points over the scene rather than sampling directions. Not being very relevant to the goal of this lecture, no further details will be discussed.

The path integral is given by

$$I_{pixel} \propto \int_{\mathcal{P}} f(\bar{p}) dp \approx \frac{1}{N_{spp}} \sum_i \frac{f(\bar{p}_i)}{p(\bar{p}_i)} \quad (3.5)$$

where $\mathcal{P}$ is the space of all possible paths. Formulating the rendering problem as a path integral problem allows leveraging results from many areas of knowledge, such as Markov chain random walks [9], Metropolis integration [10], among others, with special relevance to **BiDirectional Path Tracing** [3].

Note that some (many) paths can evaluate to zero, particularly if the deterministic connection to the light source is occluded by some geometry, as illustrated by the green path in figure 5. On the other hand, some non-zero paths (e.g., yellow path) might have low probability of being generated, resulting in a very high ratio $\frac{f(\bar{p})}{p(\bar{p})}$, thus increasing variance.

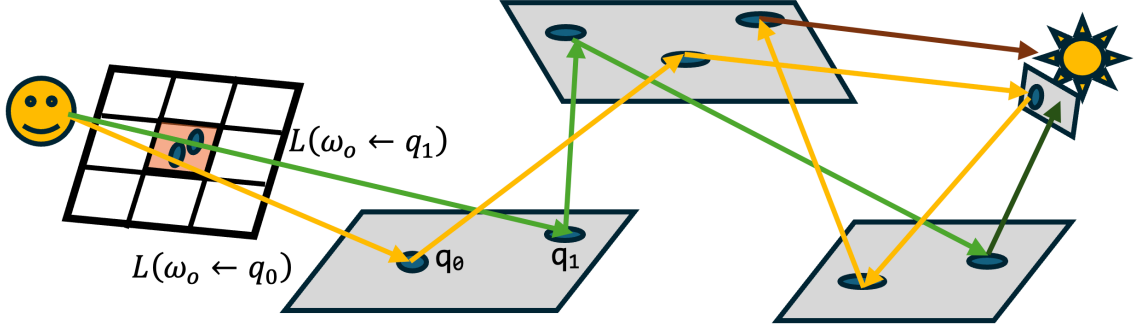


Figure 5: Path tracing - low probability path

3.3 BiDirectional Path Tracing

The **BiDirectional Reflection Distribution Function** is reciprocal [11], i.e., $f_r(q_{j-1} \leftarrow q_j \leftarrow q_{j+1}) = f_r(q_{j+1} \leftarrow q_j \leftarrow q_{j-1})$ (see [4] for a generalization to the transmissive case). Paths can therefore be traced in any direction. Section 3.1 described light transport as originating at the light source and being finally measured by the camera sensor. But the associated equations and figures depict a process starting at the camera sensor and randomly walking towards the light source. This is a result of the reciprocity of the light transport equation, allowing pixel value estimates to be computed based on both paths starting from the camera or from the light source, and, in fact, on combinations thereof.

The **BiDirectional Path Tracing (BDPT)** algorithm [3] strives to build paths between the camera and the light source by combining both forward (from the camera) and backward (from the light source) sampling.

A camera path $\bar{p}_{cam} = \{o, q_0, \dots, q_{l-1}\}$, and a light path, $\bar{p}_L = \{\dot{L}, p_0, \dots, p_{l-1}\}$, are stochastically generated, as illustrated in figure 6.

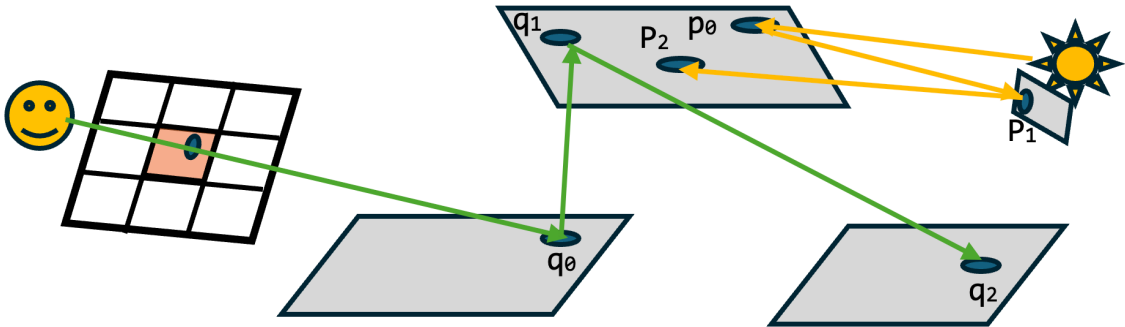


Figure 6: BiDirectional Path Tracing: forward and backward paths

New complete paths are evaluated by deterministically connecting each point $o \dots q_j$ in the camera path to each point $\dot{L} \dots p_j$ in the light path. Figure 7 illustrates the path obtained by connecting q_0 with p_0 .

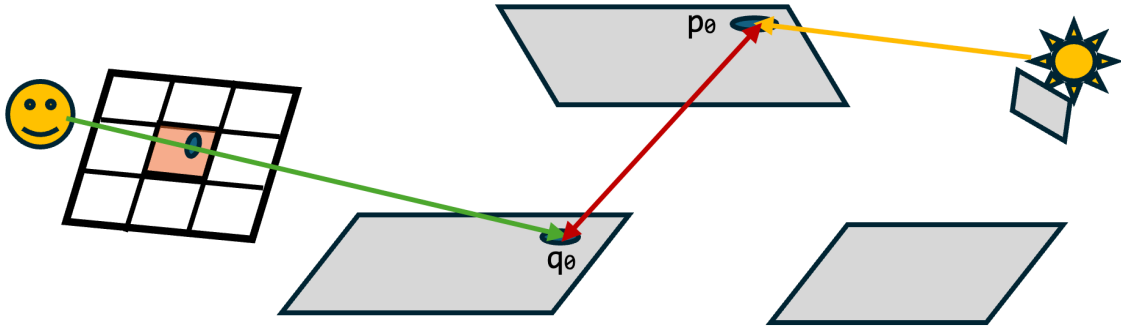


Figure 7: BiDirectional Path Tracing: connecting vertices on the forward and backward paths

The joint product of the camera and light paths results in the evaluation of a larger number of paths, increasing the probability of finding non-zero contributions. Furthermore, although the path $\{0, q_0, p_0, \dot{L}\}$ was generated by deterministically connecting q_0 with p_0 , it could have been generated in a diversity of manners (see figure 8), each with a different probability.

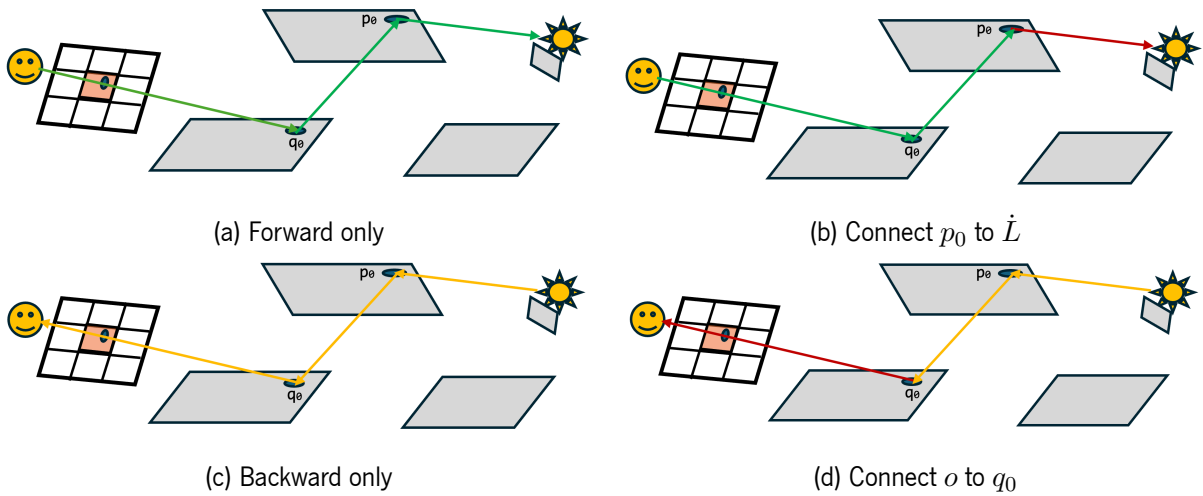


Figure 8: Different possibilities for sampling $\{0, q_0, p_0, \dot{L}\}$ (besides figure 7)

Using **Multiple Importance Sampling** (equation 2.6) to account for all the different ways in which the path could have been generated greatly reduces the variance by avoiding small denominators in equation 3.5. The example illustrated in figures 7 and 8 shows that any path $\bar{p}$ of length l could have been generated in $P = l + 2$ different manners, corresponding to forward only, plus backward only, plus having the deterministic connection at each edge. P alternative probabilities $p_t(\bar{p}), t = 0 \dots P - 1$ can therefore be factored in, weighing the path contribution as $\frac{f(\bar{p})}{\sum_{t=0}^{P-1} p_t(\bar{p})}$.

Chapter 4

Quantum Circuits and Feynman Sum of Paths

The electron does anything it likes; it goes in any direction, at any speed, forward or backward in time, however it likes – and then you add up the amplitudes and it gives you the wave function.

Some Strangeness In the Proportion

Freeman Dyson

This lecture departs from the Hamiltonian formalism of quantum mechanics, introduces layered quantum circuits and presents the Feynman sum of paths formalism as an equivalent to the Hamiltonian description.

4.1 The Hamiltonian Formalism

The Hamiltonian formalism of quantum mechanics [12] implies the existence of a vector in a complex Hilbert space that constitutes the quantum system's state, $|\psi\rangle$. The dynamics of the system, i.e., its state evolution over time, is governed by the Schrödinger equation:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle \quad (4.1)$$

where $\hat{H}$ is the Hamiltonian operator, representing the total energy of the quantum system.

A solution to the Schrödinger equation is obtained by integrating equation 4.1 over some time interval, e.g., $[0 \dots t]$:

$$|\psi(t)\rangle = e^{-i\hat{H}t/\hbar} |\psi(0)\rangle \quad (4.2)$$

The Hamiltonian is an observable of the quantum system since it represents its total energy. It must, therefore, be a Hermitian operator, implying that:

- its eigenvalues are real and correspond to the system's possible energy levels;
- its eigenvectors form an orthogonal basis, corresponding to the system's stationary states;
- $\hat{U}(t) = e^{-i\hat{H}t/\hbar}$ is a unitary operator, also referred to as the time evolution operator, modelling the quantum state evolution $|\psi(t)\rangle = \hat{U}(t)|\psi(0)\rangle$.

Within the context of quantum computation, the quantum system is composed of n quantum objects, each living in a 2-dimensional complex Hilbert space, referred to as qubits. The state vector, $|\psi\rangle$, is a $N = 2^n$ dimensional vector, each entry being a complex number corresponding to the probability amplitude of the respective basis state. The unitary operator, $\hat{U}$, is a square matrix with $2^n * 2^n = 2^{2n}$ entries. In the general case, classical representation and evaluation of the dynamics of such a quantum system are believed to require resources, both time and space, which grow exponentially with system size n [13].

A fundamental property of unitary operators is that $\hat{U}\hat{U}^\dagger = \hat{U}^\dagger\hat{U} = \mathbb{I}$, implying that quantum unitary evolution is reversible:

$$\begin{aligned}\psi(t) &= \hat{U}(t) \psi(0) \\ \hat{U}^\dagger(t) \psi(t) &= \hat{U}^\dagger(t) \hat{U}(t) \psi(0) \\ \psi(0) &= \hat{U}^\dagger(t) \psi(t).\end{aligned}\tag{4.3}$$

Reversibility can be leveraged to allow bidirectional evaluation of a quantum amplitude, similar to the **BDPT** algorithm and the reciprocity of the **BRDF**, as discussed in section 3.3.

4.2 Quantum Circuits

The unitary time evolution operator $\hat{U}$ can be decomposed into a sequence of simpler unitaries $\hat{U}_l$, corresponding to discretizing time into L contiguous subintervals. These unitaries compose via matrix multiplication, with $\hat{U} = \prod_{l=L-1}^0 \hat{U}_l$, as illustrated in figure 9 for $L = 3$. Each resulting unitary $\hat{U}_l$ is commonly referred to as a layer of the quantum circuit.

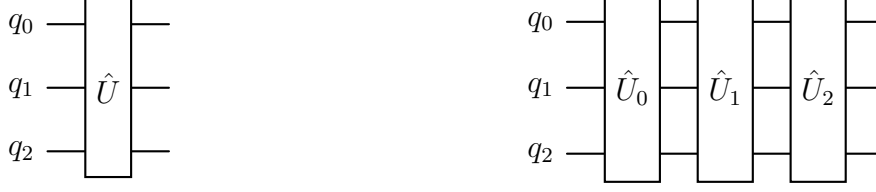


Figure 9: Unitaries are composed in time via multiplication $\hat{U} = \hat{U}_2 * \hat{U}_1 * \hat{U}_0$.

A unitary time evolution operator can also be decomposed across space, by applying different unitaries to different subsets of qubits. Composition is now performed through tensor products, with $\hat{U}_l = \bigotimes_j \hat{U}_{l,(j)}$, where j is a tuple identifying the qubits the unitary acts upon, as illustrated in figure 10.

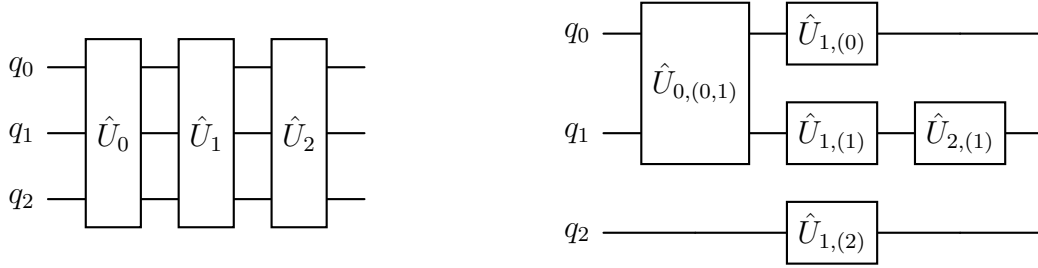


Figure 10: Unitaries are composed in space via the tensor product:

$$\hat{U}_0 = \hat{U}_{0,(0,1)} \otimes \mathbb{I}_{(2)} ; \quad \hat{U}_1 = \hat{U}_{1,(0)} \otimes \hat{U}_{1,(1)} \otimes \hat{U}_{1,(2)} ; \quad \hat{U}_2 = \mathbb{I}_{(0)} \otimes \hat{U}_{2,1} \otimes \mathbb{I}_{(2)}$$

In the circuit model of quantum computation, fundamental unitaries are represented as quantum gates. The entire circuit applies the composite unitary $\hat{U}$ to the input state, in fact evolving it in time with $\hat{U} = e^{-i\hat{H}t/\hbar}$ as modeled by the Schrödinger equation (eq. 4.2). Each line in a quantum circuit represents a qubit; the circuit must be read from left to right, corresponding to the passage of time [14].

The simulation of quantum circuits can be performed using the Schrödinger method [13]. Quantum states and unitary operators are explicitly represented as vectors and matrices. In practice, space requirements are $\mathcal{O}(2^n)$; thus, this approach is prohibitive for quantum systems of interesting sizes. There are proposals to mitigate space complexity whenever the state or the unitary exhibit structure (or redundancy). These include, for example, Decision Diagrams [15], Tensor Network Contraction [16], and Circuit Knitting [17].

4.3 Path Integral Formalism

Richard Feynman proposed in 1948 the path integral formalism of quantum mechanics [1], offering a view of quantum theory complementary to the Hamiltonian formalism (and others, such as Heisenberg's formalism [18]).

The path integral formalism dispenses with the notion of a vector during state evolution. The computational basis of an n -qubit system is a set of orthogonal vectors $|i\rangle, i = 0 \dots (2^n - 1)$, where each computational basis state $|i\rangle$ is a classical state. The transition amplitude between two classical states, $\langle i | \hat{U} | j \rangle$, is given by the integral (or sum in this discrete case) of the amplitudes of all possible classical histories of the system [2]. A history is a sequence (or path) of classical states that starts in the initial state $|j\rangle$ and ends in the final state $|i\rangle$.

A word of caution is required on how to read a classical state $|i\rangle$ in an n qubits system. Suppose there are 3 qubits; a classical state is often written in decimal, e.g., $|5\rangle$, or as a binary string $|101\rangle$, which corresponds to the (qu)bits actual values. But on an expression, such as $\langle i | \hat{U} | j \rangle$, the bra's and ket's represent state vectors, with 2^n entries. Since path integration requires only classical states, the state vector has a single entry equal to one, all others being 0; this is referred to as *one-hot encoding*. Depending on the context, the above classical state can be represented as $|5\rangle = |101\rangle = \{0, 0, 1, 0, 0, 0, 0, 0\}^T$. The transition amplitude $\langle i | \hat{U} | j \rangle$ is in fact a selector of a single entry of the unitary, given by $\langle i | \hat{U} | j \rangle = \hat{U}[i, j]$. This would be a trivial operation if one had access to $\hat{U}[i, j]$, but computing it is the real hard problem.

In the context of this lecture, we will not demonstrate the derivation of the path integral formalism from first principles (details can be found in [2, 6]), but we will derive it from the transition amplitude. Consider a quantum circuit with n qubits and L layers, initial state $|j\rangle$ and final state $|i\rangle$. Remember that

$\sum_{t_l=0}^{2^n-1} (|t_l\rangle\langle t_l|) = \mathbb{I}$, where the $|t_l\rangle$ are the computational basis states. Then we can write

$$\begin{aligned}
\langle i | \hat{U} | j \rangle &= \langle i | \prod_{l=L-1}^0 \left(\hat{U}_l \right) | j \rangle \\
&= \langle i | \prod_{l=L-1}^1 \left(\hat{U}_l \right) \hat{U}_0 | j \rangle \\
&= \langle i | \prod_{l=L-1}^1 \left(\hat{U}_l \sum_{t_l=0}^{2^n-1} (|t_l\rangle\langle t_l|) \right) \hat{U}_0 | j \rangle \\
&= \sum_{t_{L-1}=0}^{2^n-1} \cdots \sum_{t_1=0}^{2^n-1} \left[\langle i | \hat{U}_{L-1} | t_{L-1} \rangle \prod_{l=L-2}^1 \left(\langle t_{l+1} | \hat{U}_l | t_l \rangle \right) \langle t_1 | \hat{U}_0 | j \rangle \right] \\
&= \sum_{t_{L-1}=0}^{2^n-1} \cdots \sum_{t_1=0}^{2^n-1} \left[\hat{U}_{L-1}[i, t_{L-1}] \prod_{l=L-2}^1 \left(\hat{U}_l[t_{l+1}, t_l] \right) \hat{U}_0[t_1, j] \right] \\
&= \sum_{t_{L-1}=0}^{2^n-1} \cdots \sum_{t_1=0}^{2^n-1} f(\bar{p}(i, t_{L-1}, \dots, t_1, j))
\end{aligned}$$

Let $\mathcal{P}_{L,n}^{j,i}$ be the set of all possible paths of length L in an n -qubit circuit from classical state $|j\rangle$ to $|i\rangle$. This set contains $2^{n*(L-1)}$ different paths. A $L+1$ -tuple $s = (i, \{0 \dots 2^n - 1\}^{L-1}, j)$ uniquely identifies each path. The path summation equation can finally be written as given by equation 4.4.

$$\begin{aligned}
\langle i | \hat{U} | j \rangle &= \sum_{\bar{p}_s \in \mathcal{P}_{L,n}^{j,i}} f(\bar{p}(i, t_{L-1}, \dots, t_1, j)) \\
f(\bar{p}(s)) &= f(\bar{p}(i, t_{L-1}, \dots, t_1, j)) \\
&= \hat{U}_{L-1}[i, t_{L-1}] \prod_{l=L-2}^1 \left(\hat{U}_l[t_{l+1}, t_l] \right) \hat{U}_0[t_1, j]
\end{aligned} \tag{4.4}$$

If each possible classical state between layers is depicted as a circle, then figure 11a illustrates path $\bar{p}(s) = (0, 7, 3, 4, 0)$, associated with a 4-layer, 3-qubit circuit. Additionally, figure 11b illustrates all 512 possible paths defined by $\mathcal{P}_{4,3}^{0,0}$.

Equation 4.4 dispenses with the concepts of state vector and the circuit's full-time evolution unitary operator. Per-layer unitaries are, apparently, required, which would imply exponential space requirements. As briefly discussed in section 4.2 quantum circuits are most often expressed in terms of 1 and 2-qubit gates, whose unitary matrix description requires, at most, 4 or 16 complex numbers, respectively. Let $\hat{U}_l = \bigotimes_g \hat{G}_{g,(u[v])}$, $g = 0 \dots n-1$, where $\hat{G}_{(u[v])}$ is the unitary of a gate acting on qubit u or on

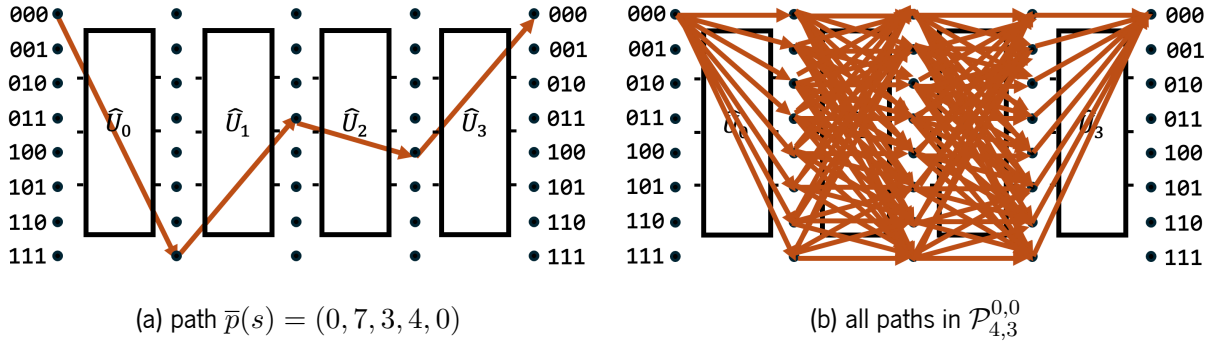


Figure 11: 3 qubits, 4 layers quantum circuit and respective paths

qubits (u, v) , with $u, v = 0 \dots n - 1$. A transition amplitude along a layer, $\langle t_{l+1} | \hat{U}_l | t_l \rangle$, is given by the product of the transition amplitudes of $\hat{U}_l$ gates:

$$\langle t_{l+1} | \hat{U}_l | t_l \rangle = \prod_g \langle t_{l+1, (u[v])} | \hat{G}_{g, (u[v])} | t_{l, (u[v])} \rangle, \quad (4.5)$$

where $t_{l, (u[v])}$ referees to addressing only qubits u and, eventually, v .

Any layer is composed by at most n gates, and each gate has an upper bound on the number of qubits it acts upon (typically 1 or 2). The path summation algorithm that results from substituting equation 4.5 in 4.4 exhibits no exponential space complexity – see algorithm 1¹. Space complexity is at most $\mathcal{O}(n * L)$, or $\mathcal{O}(L)$ if the gates' unitaries are never stored.

¹ Details on layer level methods, such as `EvalLayer()`, are provided in appendix A.

Algorithm 1: Canonical Feynman sum of paths

Data: CircuitLayers, L, n, state₀, state_f

Result: Amplitude

```
1 t[0] ← state0 ; t[1:L-1] ← 0 ; c_layer ← 0 ;
2 throughput[0] ← 1. + i0. ;
3 amplitude ← 0. + i0. ; finish ← false ;
4 while (not finish) do
5     pruneTree ← false ;
6     for (l ← c_layer . . . L - 2) and (not pruneTree) do
7         c_state ← t[l]; n_state ← t[l+1];
8         throughput[l+1] ← throughput[l] * EvalLayer (CircuitLayers, l, c_state, n_state) ;
9         if throughput[l+1] == 0 then pruneTree ← true ; upLayer ← l+1 ;
10    end
11    if not pruneTree then
12        // Last layer
13        throughput[L] ← throughput[L-1] * EvalLayer (CircuitLayers, L-1, c_state, statef) ;
14        amplitude ← amplitude + throughput[L] ;
15        upLayer ← L+1 ;
16    end
17    // Update layers' states
18    continue ← true ;
19    for (l ← upLayer . . . 1) and continue do
20        t[l] ← t[l]+1 ; c_layer ← l-1;
21        if t[l] < 2n then
22            continue ← false
23        else
24            t[l] ← 0 ; if l == 1 then finish ← true;
25        end
26    end
27 end
28 return amplitude ;
```

4.4 Discussion

Feynman's sum of paths simulation of a quantum circuit is $\mathcal{O}(L)$ in space, a major advantage over Schrödinger's simulation methods, which are $\mathcal{O}(2^n)$ in space. However, path summation approaches exhibit a time complexity exponential with both n and L , $\mathcal{O}(2^{n*(L-1)})$, hindering their use with real, interesting-sized circuits.

A variety of quantum circuits exhibits sparse path spaces, meaning that a significant fraction of the paths in $\mathcal{P}_{L,n}^{j,i}$ have zero amplitude, not contributing to the transition amplitude $\langle i|\hat{U}|j\rangle$. Tracing paths within this subset of zero amplitude paths, denoted by $\mathcal{Z}_{L,n}^{j,i} \subset \mathcal{P}_{L,n}^{j,i}$, is a computational overhead. It is, however, not clear how to identify *a priori* whether or not a path belongs to $\mathcal{Z}_{L,n}^{j,i}$.

Tables 2 and 3, which describe the circuits used in this lecture, illustrate this sparseness in the columns **#paths** and **#NZ**. The former indicates the total number of different paths connecting the initial to the final state (cardinality of $\mathcal{P}_{L,n}^{j,i}$). The latter indicates how many of these paths have a non-zero amplitude (cardinality of $\overline{\mathcal{Z}}_{L,n}^{j,i} = \mathcal{P}_{L,n}^{j,i} \setminus \mathcal{Z}_{L,n}^{j,i}$). **#NZ** could not be calculated for the larger circuits, since it would require huge computational power for relatively low-value information. However, when available, **#NZ** is dozens of orders of magnitude smaller than the total number of paths. From this data, it is reasonable to argue that larger circuits with the same structure would exhibit larger **(#paths - #NZ) / #paths** ratios. Path integration algorithms able to find more paths in $\overline{\mathcal{Z}}_{L,n}^{j,i}$ will have a clear computational advantage.

Note that the above argument does not claim that all quantum circuits induce a sparse path space ². The argument is that all circuits used in this lecture exhibit sparse path spaces, sparseness increases with circuit size within the same family of circuits, and that an algorithm's ability to find non-zero paths is determinant for performance.

Algorithm 1 prunes a tree of paths when a zero transition amplitude between layers is found (line 9). This approach's efficiency is larger when zero transitions occur in earlier layers of the circuit. Besides pruning no mechanism is included to maintain tracing within the non-zero subset of the path space, $\overline{\mathcal{Z}}_{L,n}^{j,i}$.

Global Illumination-inspired **Monte Carlo** algorithms, discussed in chapter 5, seek to efficiently evaluate robust, low-variance, estimates of $\langle i|\hat{U}|j\rangle$, while striving to keep computations within $\overline{\mathcal{Z}}_{L,n}^{j,i}$.

² The measure of sparseness being used is the ratio **(#paths - #NZ) / #paths**, which treats the paths' amplitudes as binary (zero vs. non-zero). Other metrics, such as the Gini index [19], take into account the actual magnitudes of the paths' amplitudes, which might be useful in scenarios with many small contributing paths but dominated by a few larger contributing paths, such as with open quantum systems, i.e., in the presence of noise.

Chapter 5

Path Integration Monte Carlo

Monte Carlo is not about certainty, but rather embracing randomness to shine light over the patterns putatively obfuscated by complexity.

The transition amplitude $\langle i | \hat{U} | j \rangle$ associated with a quantum circuit with L layers and n qubits can be computed as a sum of the amplitudes of all classical paths from classical initial state $|j\rangle$ to classical final state $|i\rangle$:

$$\begin{aligned} \langle i | \hat{U} | j \rangle &= \sum_{t_{L-1}=0}^{2^n-1} \cdots \sum_{t_1=0}^{2^n-1} f(\bar{p}(i, t_{L-1}, \dots, t_1, j)) \\ &= \sum_{\bar{p}(s) \in \mathcal{P}_{L,n}^{j,i}} f(\bar{p}(s)) \end{aligned} \tag{5.1}$$

where $\mathcal{P}_{L,n}^{j,i}$ is the set of all such paths. Algorithm 1 can be used to naively traverse the tree of paths.

Monte Carlo quadratures can be used to estimate $\langle i | \hat{U} | j \rangle$. N paths are stochastically sampled from $\mathcal{P}_{L,n}^{j,i}$; the estimate is the weighted average of these samples' amplitudes:

$$\langle i | \hat{U} | j \rangle \approx \frac{1}{N} \sum_{u=0}^{N-1} [w(\bar{p}(s_u)) f(\bar{p}(s_u))] , \text{ with } \bar{p}(s_u) \in \mathcal{P}_{L,n}^{j,i} \tag{5.2}$$

The sampling process and the weights $w(\bar{p}(s_u))$ define the algorithm. This lecture proposes using Monte Carlo algorithms inspired by light transport simulation techniques, as described in chapter 3. The analogy involves denoting the initial state with the light source and the final state as the observer. The state evolves through time from left to right, such as light propagates in space from the source to the camera - figure 12.

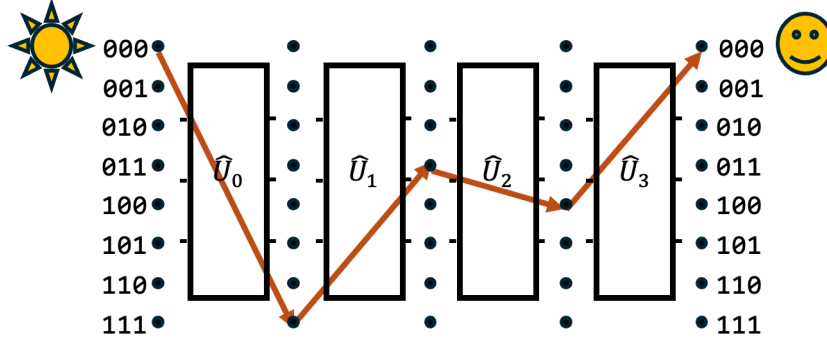


Figure 12: Analogy of information propagation on a quantum circuit with light transport.

This chapter describes four Monte Carlo algorithms, hopefully in order of increasing convergence rate: forward uniform sampling path tracing, forward importance sampling path tracing, bidirectional path tracing, and multiple importance sampling bidirectional path tracing.

5.1 Forward Uniform Sampling Path Tracing

Each path $\bar{p}_s = \bar{p}(i, t_{L-1}, \dots, t_1, j)$ is built by uniform sampling each intermediate's layer state $t_l, l = 1 \dots L-1$ from the range of all possible states for n qubits: $0 \dots 2^n - 1$. Each connection among layers is therefore generated with probability $1/2^n$, with the state t_{L-1} being deterministically connected to the final state $|i\rangle$ across the last layer $L-1$, i.e., this connection has probability 1. Every path has thus the same probability of being generated: $p(\bar{p}_s) = 1/2^{n*(L-1)}$.

The Monte Carlo weights are constant, $w(\bar{p}(s_u)) = 1/p(\bar{p}(s_u)) = 2^{n*(L-1)}$ and the Monte Carlo estimator is given by

$$\langle i | \hat{U} | j \rangle \approx \frac{2^{n*(L-1)}}{N} \sum_{u=0}^{N-1} f(\bar{p}(s_u))$$

Algorithm 2 details this approach in pseudo-code ¹.

Algorithm 2: Forward Uniform Sampling Path Tracing

Data: CircuitLayers, L , n , state_0 , state_f , N

Result: Amplitude

```

1   $t[0] \leftarrow \text{state}_0$  ;  $t[L] \leftarrow \text{state}_f$  ;
2   $\text{amplitude} \leftarrow 0.$  +  $i0.$  ;  $\text{sample} \leftarrow 0$  ;
3  while ( $\text{sample} < N$ ) do
4       $\text{throughput}[0] \leftarrow 1.$  +  $i0.$  ;  $\text{isZero} \leftarrow \text{false}$  ;
5      for ( $l \leftarrow 0 \dots L - 1$ ) and ( $\text{not isZero}$ ) do
6          if  $l < L-1$  then  $t[l+1] \leftarrow \text{rand}(0 \dots 2^n)$ ;           // sample next state
7           $c\_state \leftarrow t[l]$ ;  $n\_state \leftarrow t[l+1]$ ;
8           $\text{throughput}[l+1] \leftarrow \text{throughput}[l] * \text{EvalLayer}(\text{CircuitLayers}, l, c\_state, n\_state)$  ;
9          if  $\text{throughput}[l+1] == 0$  then  $\text{isZero} \leftarrow \text{true}$  ;      // early path termination
10     end
11     if not isZero then  $\text{amplitude} \leftarrow \text{amplitude} + \text{throughput}[L]$ ;
12      $\text{sample}++$  ;
13 end
14  $\text{amplitude} \leftarrow \frac{2^{n*(L-1)}}{N} * \text{amplitude}$  ;           // weighed average
15 return  $\text{amplitude}$  ;
```

Since the quantum circuit's structure, i.e., its gates, is ignored in the sampling process, no effort is made to sample within the non-zero subset of the path space, $\overline{\mathcal{Z}}_{L,n}^{j,i}$. If the path space is sparse, then a vast majority of the paths will have a contribution $f(\overline{p}(s_u))$ equal to zero, resulting in wasted computation time and large variance.

5.2 Forward Importance Sampling Path Tracing

Each path is generated across $L - 1$ layers, from the initial state j to the next-to-last state, t_{L-1} . The classical state at each intermediate layer is sampled from a probability distribution defined by the gates of that layer – details on how to sample the next classical state t_{l+1} given t_l and the gates in layer l are presented in section 5.2.1. The path segment sampled from $|j\rangle$ to $|t_{L-1}\rangle$ is necessarily a non-zero

¹ Details on EvalLayer() are provided in appendix A.

amplitude segment, with non-zero probability, since it was generated accordingly to the gates' unitaries – see figure 13. The last connection, from $|t_{L-1}\rangle$ to $|i\rangle$ across $\hat{U}_{L-1}$, is deterministic (probability 1, since $|i\rangle$ is fixed); however, its amplitude can be (and often will be) 0, since $\hat{U}_{L-1}$ can imply $\langle i | \hat{U}_{L-1} | t_{L-1} \rangle = 0$.

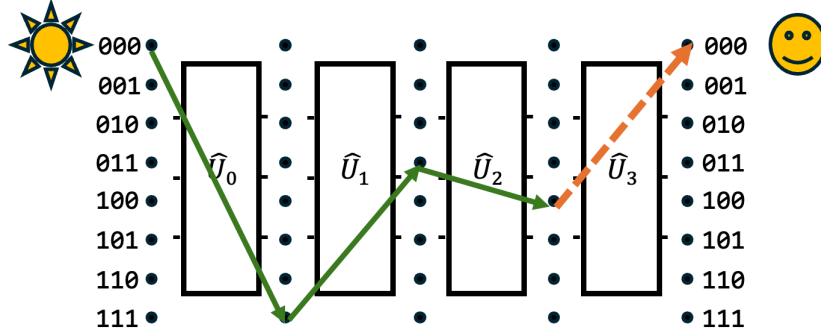


Figure 13: Forward importance sampling path tracing: up to layer $L - 1$ (green arrows) the path is guaranteed to have both non-zero amplitude and probability. The deterministic connection to the final state (orange dashed arrow) has probability 1, but its amplitude $\langle i | \hat{U}_{L-1} | t_{L-1} \rangle$ can be zero.

Each path is sampled with probability $p(\bar{p}(s_u))$; the weights are given by $w(\bar{p}(s_u)) = 1/p(\bar{p}(s_u))$ and the estimator becomes $\langle i | \hat{U} | j \rangle \approx \frac{1}{N} \sum_{u=0}^{N-1} \frac{f(\bar{p}(s_u))}{p(\bar{p}(s_u))}$.

Forward importance sampling convergence rate depends on:

- how hard is it to connect forward reachable states $|t_{L-1}\rangle$ to $|i\rangle$. If $\hat{U}_{L-1}$ has a large branching factor (e.g., all Hadamard gates, in the limit), then the final state is reachable with high probability (probability 1 in the limit);
- how likely it is to forward sample a path segment leading to a next-to-last state, $|t_{L-1}\rangle$, which connects to the final state with non-zero amplitude. If such segments are very unlikely, then the non-zero paths have very low probability. When they occur, their contribution is huge, since the amplitude is divided by a small probability ($\ll 1$), resulting in a high variance Monte Carlo estimate.

Algorithm 3 details this approach in pseudo-code ².

Algorithm 3: Forward Importance Sampling Path Tracing

Data: CircuitLayers, L, n, state₀, state_f, N

Result: Amplitude

```

1  t[0] ← state0 ; t[L] ← statef ;
2  amplitude ← 0. + i0. ; sample ← 0 ;
3  while (sample < N) do
4      throughput[0] ← 1. + i0. ; isZero ← false ;
5      for (l ← 0 ... L - 1) and (not isZero) do
6          pNextState ← 1 ;
7          if l < L-1 then // sample next state
8              t[l+1], pNextState ← SampleLayer (CircuitLayers, l, c_state);
9          end
10         c_state ← t[l]; n_state ← t[l+1];
11         layer_f ← EvalLayer (CircuitLayers, l, c_state, n_state) ;
12         if layer_f==0 or pNextState==0 then // early path termination
13             isZero ← true;
14         else
15             throughput[l+1] ← throughput[l] *  $\frac{\text{layer\_f}}{\text{pNextState}}$ ;
16         end
17     end
18     if not isZero then amplitude ← amplitude + throughput[L];
19     sample++ ;
20 end
21 amplitude ←  $\frac{\text{amplitude}}{N}$  ; // weighed average
22 return amplitude ;

```

5.2.1 Importance Sampling the Next State

A fundamental operation of all Monte Carlo path tracing algorithms is to generate the path's next vertex t_{l+1} , given the current classical state t_l and the quantum circuit's next layer of gates $\hat{U}_l$.

² Details on layer level methods, EvalLayer() and SampleLayer(), are provided in appendix A.

A layer, in the context of this lecture, has a maximum depth of one gate, with each gate operating on a subset of 1 or 2 qubits. The gates' unitaries, $\hat{G}$, compose through tensor products, with $\hat{U}_l = \bigotimes_j \hat{G}_{l,(j)}$, where j is a tuple identifying the qubits the gate acts upon. Since each layer has a maximum depth of one, within a layer each qubit is acted upon, at most, by a single gate.

The sum of paths formulation of quantum mechanics implies that the layer's input state $|t_l\rangle$ is a classical state, i.e., a vector with 2^n entries, all zero, except the one corresponding to the $|t_l\rangle$ basis state, which is equal to one. Since $|t_{l+1}\rangle \propto \hat{U}_l |t_l\rangle$, t_l acts as a column selector in $\hat{U}_l$:

$$\begin{pmatrix} t_{l+1,(0)} \\ \vdots \\ t_{l+1,(2^n-1)} \end{pmatrix} \propto \begin{bmatrix} U_{l,(0,0)} & \dots & U_{l,(0,t_l)} & \dots & U_{l,(0,2^n-1)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ U_{l,(2^n-1,0)} & \dots & U_{l,(2^n-1,t_l)} & \dots & U_{l,(2^n-1,2^n-1)} \end{bmatrix} \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

$|t_{l+1}\rangle$ is also required to be a classical state, thus the equality symbol is not used in the previous equation. Rather, the vector given by the square amplitude of each element of column $U_{l,(:,t_l)}$ is used as a probability distribution to sample an actual classical state for $|t_{l+1}\rangle$:

$$p(t_{l+1}) = \left(|U_{l,(0,t_l)}|^2 \quad \dots \quad |U_{l,(2^n-1,t_l)}|^2 \right)^T$$

Since the layer's unitary is given by a tensor product of gates and each qubit is acted upon by a single gate, evaluating $\hat{U}_l$ is not required (this would be exponential on the number of qubits). $|t_{l+1}\rangle$ can be generated on a per-gate basis. Gate $\hat{G}_{l,(j)}$ acts on the qubits identified by the tuple (j) . The corresponding qubits within $|t_l\rangle$ are used to select the correct column within $\hat{G}_l$ and the associated probability distribution can be used to generate bits (j) of $|t_{l+1}\rangle$.

Figure 14 depicts one layer of a 4-qubit circuit that will be used as an example to importance sample the next state.

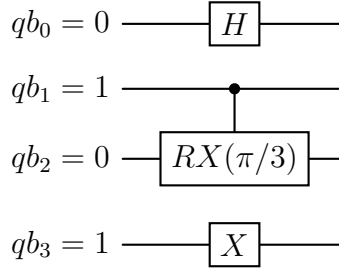


Figure 14: A typical layer to sample a classical state $|t_{l+1}\rangle$ given $|t_l\rangle$

Qubit qb_0 is acted upon by a Hadamard gate, whose unitary is

$$2^{-\frac{1}{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Since $qb_0 = 0$, the first column is selected and the probability distribution over the output is $p(t_{l+1,(0)}) = (1/2, 1/2)^T$, with output qubit qb_0 being selected as either 0 or 1 with identical probability equal to 0.5.

Qubits qb_1 and qb_2 are acted upon by a conditional X rotation gate with parameter $\pi/3$, whose unitary is

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \sqrt{3}/2 & -j/2 \\ 0 & 0 & -j/2 & \sqrt{3}/2 \end{bmatrix}$$

Since $qb_1 = 1, qb_2 = 0$, column $10_{b_2} = 2$ is selected and the output is sampled from $\{10, 11\}$ with a probability distribution given by $(|\sqrt{3}/2|^2, |-j/2|^2)^T = (3/4, 1/4)^T$.

Qubit qb_3 is acted upon by a NOT gate, whose unitary is

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

This is a deterministic, non-branching gate. Since $qb_3 = 1$, the second column is selected and the output is $qb_3 = 0$ with probability 1.

The joint probability distribution over the output state does not have to be represented (it would be exponential in the number of qubits). For this example, it would be as depicted in table 1.

Output	Probability
qb_3, qb_2, qb_1, qb_0	
0010	$3/4 * 1/2 = 3/8$
0011	$3/4 * 1/2 = 3/8$
0110	$1/4 * 1/2 = 1/8$
0111	$1/4 * 1/2 = 1/8$

Table 1: Output states and respective probability for the example presented in figure 14.

5.3 Bidirectional Path Tracing

A forward path segment (from $|j\rangle$ to $|t_{L-1}\rangle$) and a backward path segment (from $|i\rangle$ to $|t_1\rangle$) are stochastically sampled using importance sampling as described in section 5.2³. Figure 15 presents an example of sampled paths for $L = 4$. Each of these path segments is guaranteed to have both amplitude and probability different from zero, since they were sampled using the gates' unitaries.

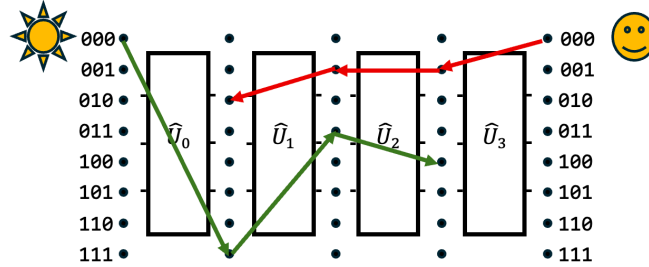


Figure 15: Forward and backward importance sampled path segments for $L = 4$.

L full paths are built by deterministically connecting (probability equal to 1), per path, state t_l to state t_{l+1} , with $l = 0 \dots L - 1$ and $t_0 = j$, $t_L = i$. Figure 16 illustrates the paths built from the example depicted in the previous figure.

The potential advantage of **BDPT** stems from the fact that each of these L paths is more likely to have a non-zero contribution than forward only paths. Algorithm 4 depicts this approach in pseudo-code.

³ For backward sampling the conjugate transpose of the gates' unitaries must be used.

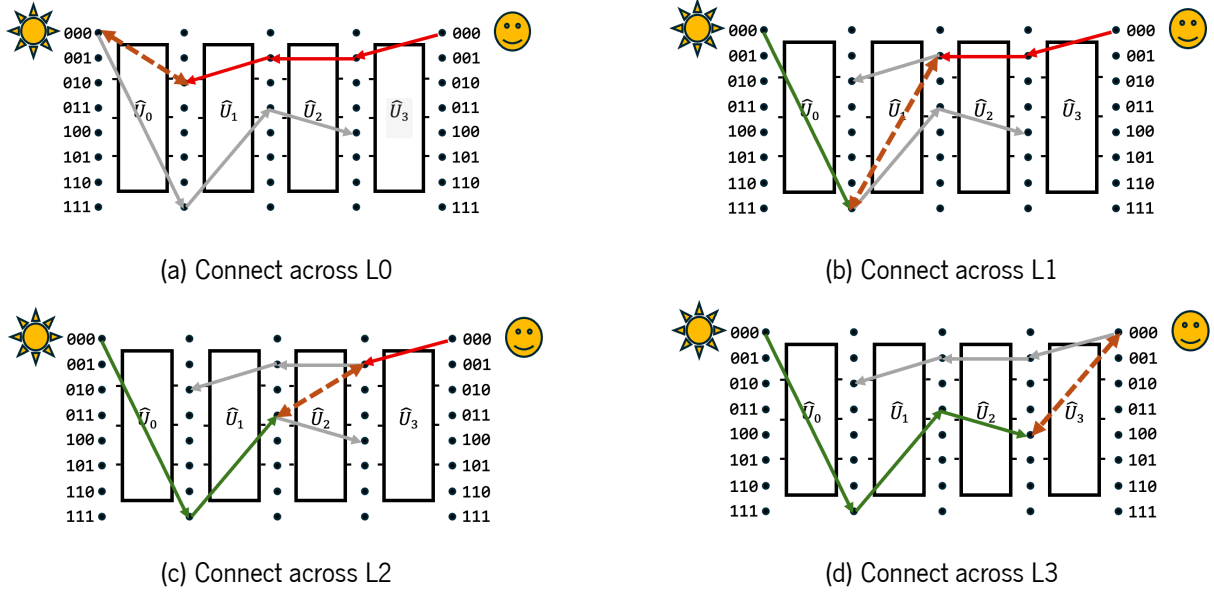


Figure 16: Paths generated by connecting forward and backward path segments. The dashed dark orange arrow represents the deterministic connection. Non-used path segments are dimmed to grey.

Algorithm 4: Bidirectional Path Tracing

Data: CircuitLayers, L , n , $state_0$, $state_f$, N

Result: Amplitude

```

1  $t[0] \leftarrow state_0$  ;  $t[L] \leftarrow state_f$  ;
2  $amplitude \leftarrow 0. + i0.$  ;  $sample \leftarrow 0$  ;
3 while ( $sample < N$ ) do
    // pF[l] stores the aggregated probability from  $j$  to  $t_l$ 
    // aF[l] stores the aggregated amplitude from  $j$  to  $t_l$ 
4    $pathF, pF, aF \leftarrow GenPathForward (CircuitLayers, L, n, state_0)$  ;
    // pB[L-1-1] stores the aggregated probability from  $i$  to  $t_l$ 
    // aB[L-1-1] stores the aggregated probability from  $i$  to  $t_l$ 
5    $pathB, pB, aB \leftarrow GenPathBackward (CircuitLayers, L, n, state_f)$  ;
6   for ( $l \leftarrow 0 \dots L - 1$ ) do
7      $ampC \leftarrow Connect (CircuitLayers, l, pathF, pathB)$  ;
8      $throughput \leftarrow aF[l] * ampC * aB[l]$  ;
9      $prob \leftarrow pF[l] * 1.0 * pB[l]$  ;
10     $amplitude \leftarrow amplitude + \frac{throughput}{prob}$  ; // weighed average
11  end
12   $sample++$  ;
13 end
14  $amplitude \leftarrow \frac{amplitude}{N * L}$  ; // weighed average
15 return  $amplitude$  ;

```

5.4 Multiple Importance Sampling Bidirectional Path Tracing

Each path in **BDPT** is built by deterministically connecting a forward path segment with a backward path segment across a layer. This same path could have been generated by deterministically connecting across any other layer.

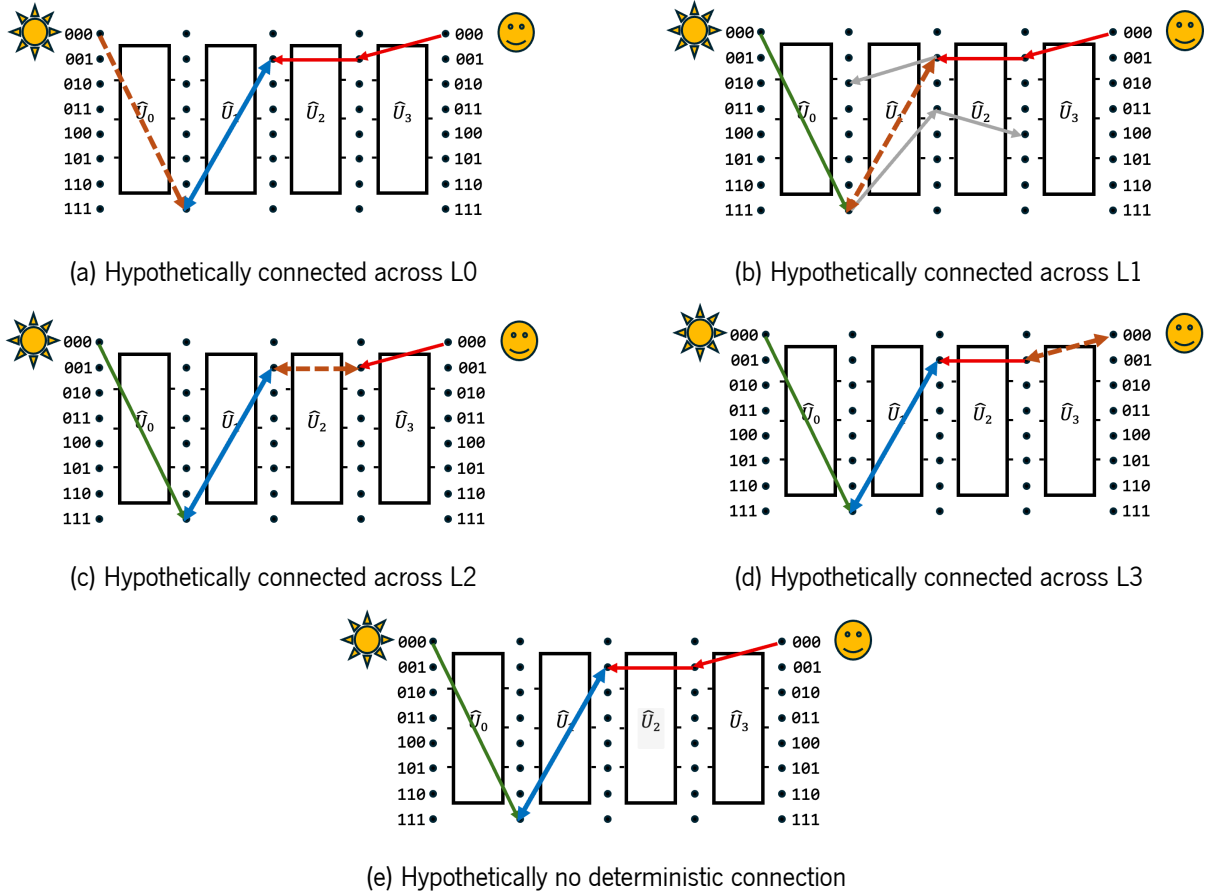


Figure 17: $L + 1$ different manners the same path, originally deterministically connected across L1 as depicted by the blue arrow (see figure 16b), could have been generated by shifting the deterministic connection (dark orange dashed arrow).

Therefore, there are $P = L + 1$ different manners each path could have been generated, each with probability $p_t(\bar{p})$, $t = 0 \dots P - 1$. Figure 17 uses the path depicted in figure 16b and effectively created by a deterministic connection across $\hat{U}_1$ and illustrates the 5 different ways the same path could have been generated.

The weights are given by $w(\bar{p}) = 1 / \sum_{t=0}^{P-1} p_t(\bar{p})$ and the estimator becomes

$$\langle i | \hat{U} | j \rangle \approx \frac{1}{N * L} \sum_{u=0}^{N-1} \sum_{l=0}^{L-1} \frac{f(\bar{p}(s_{u,l}))}{\sum_{t=0}^{P-1} p_t(\bar{p}(s_{u,l}))}$$

.

Algorithm 5 illustrates the approach in pseudo-code ⁴. Each of the L paths built by **BDPT** (loop in line 6), requires the sum of $L + 1$ different path generation probabilities (loop in line 11). The probability that the connection from t_l to t_{l+1} would have been stochastically sampled (instead of being deterministic) is evaluated in line 8.

The **for** loop in line 11 evaluates the probabilities of all alternatives where the deterministic connection could have occurred. The loop counter ll iterates over all layers ($0 \dots L - 1$), substituting a stochastic connection by a deterministic connection ($p = 1$) across $\hat{U}_{ll}$, as exemplified in figures 17a to 17d, respectively. The last iteration of the loop ($ll = L$, figure 17e) handles the case where the whole path would have been generated stochastically.

⁴ Details on `EvalLayerProb()` are provided in appendix A.

Algorithm 5: Multiple Importance Sampling Bidirectional Path Tracing

Data: CircuitLayers, L , n , $state_0$, $state_f$, N **Result:** Amplitude

```
1  $t[0] \leftarrow state_0$  ;  $t[L] \leftarrow state_f$  ;
2 amplitude  $\leftarrow 0.$  +  $i0.$  ; sample  $\leftarrow 0$  ;
3 while (sample <  $N$ ) do
    // pF[l] stores the aggregated probability from  $j$  to  $t_l$ 
    // aF[l] stores the aggregated amplitude from  $j$  to  $t_l$ 
4    pathF, pF, aF  $\leftarrow$  GenPathForward (CircuitLayers,  $L$ ,  $n$ ,  $state_0$ ) ;
    // pB[L-1-1] stores the aggregated probability from  $i$  to  $t_l$ 
    // aB[L-1-1] stores the aggregated probability from  $i$  to  $t_l$ 
5    pathB, pB, aB  $\leftarrow$  GenPathBackward (CircuitLayers,  $L$ ,  $n$ ,  $state_f$ ) ;
6    for ( $l \leftarrow 0 \dots L - 1$ ) do
7        ampC  $\leftarrow$  Connect (CircuitLayers,  $l$ , pathF, pathB) ;
8        thisConnectProb  $\leftarrow$  EvalLayerProb (CircuitLayers,  $l$ , pathF[l], pathB[l+1]) ;
9        throughput  $\leftarrow$  aF[l] * ampC * aB[l] ;
10       sumProb  $\leftarrow 0$  ;
11       for ( $ll \leftarrow 0 \dots L$ ) do
12           prob  $\leftarrow$  ThisHypothesisProb ( $l$ ,  $ll$ , pF, pB, thisConnectProb) ;
13           sumProb  $\leftarrow$  sumProb + prob ;
14       end
15       amplitude  $\leftarrow$  amplitude +  $\frac{\text{throughput}}{\text{sumProb}}$  ;           // weighed average
16   end
17   sample++ ;
18 end
19 amplitude  $\leftarrow \frac{\text{amplitude}}{N * L}$  ;           // weighed average
20 return amplitude ;
```

Multiple Importance Sampling plays a fundamental role in increasing **BDPT**'s convergence rate. Using a sum of probabilities in the estimator's denominator steers, in most cases, the computation away from divisions by small probabilities; these would result in a large ratio, therefore introducing peaks in the variance. Practical applications of **BDPT** algorithms should always perform **MIS**.

5.5 Practical Considerations

This section briefly discusses some practical technical considerations regarding the simulators' implementation. Parallelism and representation of numbers of an arbitrarily large range and precision are addressed.

Parallel processing

Samples of the proposed **Monte Carlo** algorithms can be evaluated independently of each other, allowing workload decomposition across samples and enabling exploitation of parallel computing at multiple levels simultaneously:

distributed memory, computing nodes parallelism – the workload is decomposed into large tasks consisting of sets of samples, with tasks being distributed across the multiple computing nodes allocated for the job's execution;

thread parallelism – tasks assigned to the computing nodes are further decomposed into subtasks, which are distributed across threads executing in the multiple processing cores of the computing node;

vector parallelism – within a task, samples are grouped into packets whose cardinality is equal to the processing core vector execution units width; the compiler ensures that samples of the same packet are executed using vector instructions whenever possible.

This multilevel parallel approach exhibits large weak scalability, given the embarrassingly parallel nature of the **MC** algorithms.

Representation of large range, high precision numbers

The basis states of quantum circuits with up to 64 bits are represented using C++ `unsigned long long`, which are 64-bits wide integers; each bit is used to represent the classical state of a single qubit. A path is a sequence of $L + 1$ such integers. Integer arithmetic over this datatype is supported by the processor's hardware, making it a very convenient choice, performance-wise. However, this approach is

no longer valid for circuits with more than 64 qubits. Sequences of many bits are required to represent the basis states.

Additionally, as the number of qubits increases, the paths' amplitudes and probabilities concentrate exponentially next to zero. The IEEE 754 standard for floating point representation, supported in hardware by most processors, defines single and double precision formats. The smallest absolute values supported are 2^{-126} and 2^{-1022} (normalized), and the precisions are 2^{-24} and 2^{-53} , respectively. These are clearly unable to represent the required range and precision as the number of qubits grows to the hundreds or thousands (see section 6.2.3 for simulations with 256, 1024 and 4096 qubits).

The simulators developed within the context of this lecture use "GMP - The GNU Multiple Precision Arithmetic Library" ⁵ to provide support for arbitrary-length integers and floating point numbers. This library is used when the quantum circuit width exceeds 64. Using this tool, instead of standard hardware supported formats, strongly penalizes execution times, since arithmetic operations are performed in software. Additionally, using GMP precludes using the vector processing units to simultaneously process a packet of samples within each thread.

⁵ <https://gmplib.org/>

Chapter 6

Experimental Testing

Every genuine test of a theory is an attempt to falsify it, or to refute it.
Testability is falsifiability; [...]

Conjectures and Refutations: The Growth of Scientific Knowledge

Karl Popper

This lecture hypothesizes that Bidirectional **Monte Carlo (MC)** methods, inspired by current practices in **Global Illumination**, achieve better convergence than forward-only **MC** approaches when estimating transition amplitudes associated with quantum circuits.

This chapter describes the experiments conducted to test this hypothesis and presents the associated quantitative results.

6.1 Methodology

Experimental results are generated by executing two simulators on a set of quantum circuits. The simulators implement the Forward Importance Sampling Path Tracing and the Multiple Importance Sampling Bidirectional Path Tracing algorithms, labelled **PT** and **BDPT**, respectively.

Monte Carlo's convergence is measured by the variance of the estimator, given by

$$Var(\langle I \rangle) = \frac{1}{E} \sum_{e=0}^{E-1} (I - \langle I \rangle_e)^2 \quad (6.1)$$

where E is the number of experiments, I is the true value of the integral and $\langle I \rangle_e$ is the estimate of the e^{th} experiment. This chapter reports variance for $E = 5$.

The transition amplitude estimated by the Monte Carlo process is modelled as a complex random variable, whose variance is a positive real number, equal to the sum of the variances of the real and imaginary parts:

$$\text{Var}(c) = \text{Var}(\Re(c)) + \text{Var}(\Im(c)), \quad c \in \mathbb{C}$$

To compare the two algorithms' convergence rate three different metrics are used:

variance as a function of the number of MC samples – for the **PT** algorithm one sample corresponds to one path, whereas for **BDPT** one sample is a pair of (forward, backward) path segments, from which L paths are built. For both cases, variance is expected to decrease with the number of samples and **BDPT** is expected to achieve a lower variance than **PT** for the same number of samples;

variance as a function of execution time – this is dependent on the implementation, technology, and computing resources; to allow for comparisons between the algorithms for the same circuit, simulations for any circuit are always executed with the same computing resources (processing cores, threads, memory);

efficiency as a function of the number of samples – where efficiency is a metric that combines variance and time and is given by $\frac{1}{\text{Var} \cdot \text{time}}$.

All workloads were executed in the x86 partition of the Deucalion supercomputer ¹, under project 2024.07005. CPCA. This partition consists of 500 computing nodes, each with two AMD EPYC 7742 processors, totalling 128 processing cores and 256 GB DDR4 per node.

6.1.1 Circuits

To compare the **PT** and **BDPT** algorithms' performance, data is collected using four different families of circuits, representative of different features and challenges concerning quantum state evolution. These families are:

Efficient SU2 (label **SU**) – these circuits consist of a heuristic pattern used to prepare a state for variational quantum algorithms. It is the most commonly used *ansatz* in contexts where the variational

¹ [Deucalion - RNCA supercomputer](#)

algorithm is problem-agnostic and, hopefully, hardware efficient. The pattern is composed of multiple layers of single-qubit gates spanned by $SU(2)$ and CX entanglements. $SU(2)$ is the special unitary group of degree 2; its elements are 2×2 unitary matrices with determinant 1, such as the Pauli rotation gates [20, 21].

Instantaneous Quantum Polynomial (label **IQP**) – these circuits implement the kernel inversion test for two vectors encoded as quantum states using the IQP kernel. The overlap between the two vectors is given by the probability of measuring $|0\rangle$. It has been shown that if IQP circuits can be classically simulated efficiently, then this implies the collapse of the Polynomial Hierarchy, which is deemed to be very unlikely [22, 23]. In other words, IQP circuits are most likely hard to simulate classically, which makes them an interesting challenge for the proposed algorithms.

Random (label **RND**) – these circuits are generated by randomly sampling gates from the Clifford+T gate set, according to the rules defined in [24]. The associated time evolution operator prepares a quantum state $|\psi\rangle$ that follows a Thomas-Porter distribution, i.e., most bit strings in $|\psi\rangle$ have measuring probability that is exponentially small in the circuit's number of qubits, but some strings do have a large associated probability. These circuits are known to be exponentially hard to simulate classically (in the ideal, non-noisy, case) and have been used to demonstrate quantum supremacy [25].

Hidden Shift (label **HSC**) – **Hidden Shift Circuits (HSC)** are deterministic, meaning that they prepare a classical state which is known *a priori* [26]. The output state vector prepared by a **HSC**, contains a single non-zero entry corresponding to the known binary string $\mathbf{s}$ encoded in the circuit: $p(\mathbf{s} | |\psi\rangle) = 1$. They are useful to verify the outcome of a simulator against the correct, known, output. This family of circuits can be classically simulated in polynomial time [27].

For each of these families 3 circuits of different sizes are used in the experiments. These differ in the number of qubits and circuit depth to allow for benchmarking the proposed algorithms against these features. Table 2 presents detailed data on each circuit ².

² The circuits data and descriptions are available [here](#).

Family	ID	n	L	#paths	#NZ	transition	amplitude
SU	SU5	5	8	3E10	32	$\langle 0 \hat{U} 0\rangle$	$-0.00172 + 0.00128j$
	SU15	15	22	7E94	$\approx 6E8$	$\langle 0 \hat{U} 0\rangle$	$0.00365 + 0.00061j$
	SU25	25	32	2E233	n.a.	$\langle 0 \hat{U} 0\rangle$	$-0.00012 + 0.00007j$
IQP	IQP5	5	17	1E24	32	$\langle 0 \hat{U} 0\rangle$	$-0.10446 + 0.10669j$
	IQP15	15	55	7E243	$\approx 3E4$	$\langle 0 \hat{U} 0\rangle$	$-0.00128 - 0.00009j$
	IQP25	25	95	1E830	n.a.	$\langle 0 \hat{U} 0\rangle$	$-0.00013 - 0.00022j$
RND	RND8	8	9	2E19	512	$\langle 0 \hat{U} 0\rangle$	$-0.04419 + 0.10669j$
	RND10	10	13	1E43	$\approx 2E5$	$\langle 0 \hat{U} 0\rangle$	$0.02210 + 0.05335j$
	RND12	12	15	1E61	$\approx 1E9$	$\langle 16 \hat{U} 0\rangle$	$0.00915 - 0.02210j$
HSC	HSC256	256	43	1E3600	n.a.	$\langle 1023 \hat{U} 0\rangle$	1.0
	HSC1024	1024	43	1E14400	n.a.	$\langle 1023 \hat{U} 0\rangle$	1.0
	HSC4096	4096	43	1E57400	n.a.	$\langle 1023 \hat{U} 0\rangle$	1.0

Table 2: Circuits used in experimental testing (**n** - qubits, **L** - layers, **#paths** - total number of paths, **#NZ** - number of non-zero paths). Evaluating the number of different non-zero paths requires the execution of some deterministic algorithm that iterates through all possible paths that are not pruned. This is a computationally very demanding task; therefore, values are not available (n.a.) for larger circuits.

The algorithms' scalability with circuit size is evaluated using 5 random square circuits (width equal to depth, i.e., same number of qubits and layers). To generate the circuits, gates are drawn from the Clifford plus T set, with the Hadamard gate having a 0.3 probability. Table 3 presents detailed data on each random square circuit.

ID	n	L	#paths	#NZ	transition	amplitude
RSQ8	8	8	2^{56}	2^4	$\langle 0 \hat{U} 0\rangle$	$0.15089 + 0.0625j$
RSQ12	12	12	2^{132}	2^{19}	$\langle 0 \hat{U} 0\rangle$	$-0.06440 - 0.02667j$
RSQ16	16	16	2^{240}	n.a.	$\langle 0 \hat{U} 0\rangle$	$-0.00162 - 0.00391j$

Table 3: Square Random circuits used to evaluate scalability (**n** - qubits, **L** - layers, **#paths** - total number of paths, **#NZ** - number of non-zero paths).

6.2 Results

Empirical data were collected using the two algorithms for the different circuits. The analysis of the results below is organized into a comparison of algorithms' convergence, scalability analysis, and handling of very large circuits.

To facilitate the visual interpretation of the figures, data obtained for the **PT** algorithm is presented in shades of red, whereas data for **BDPT** is presented in shades of green - the rationale being that **BDPT** is expected to display better results (green) than **PT** (red). The saturation of the respective hue represents the size of the circuit associated with each curve: smaller circuits are depicted with less saturated colors.

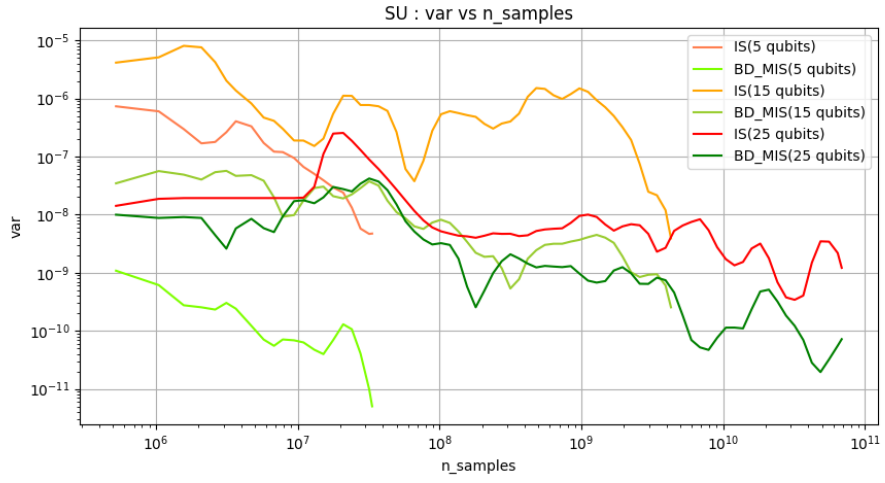
6.2.1 Convergence Comparison

Figure 18 illustrates how variance varies with the number of **MC** samples, for the range of circuits considered and both algorithms. **BDPT** consistently achieves lower variance than **PT** for the same number of samples; this is more evident for larger circuits. **BDPT** outperforms **PT** for similar sampling counts, and this trend gets more significant as circuits get larger. For the **RND12** and **IQP25** circuits, **BDPT**'s variance is smaller than **PT**'s by 3 and 6 orders of magnitude, respectively. It must be kept in mind, however, that each **BDPT** sample gives rise to L paths.

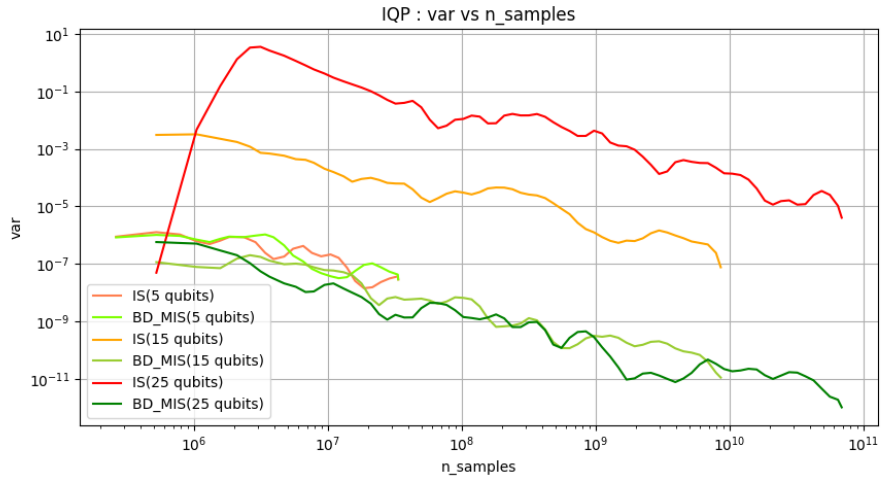
Figure 19 illustrates how variance varies with execution time. However, execution time for the shallower circuits (**SU5**, **IQP5**, **RND8**, and **RND10**) is so short that the collected data points do not capture any meaningful behaviour; data is shown only for the remaining larger circuits.

It is clear from these curves that **BDPT** takes longer to execute than **PT** for the same number of samples – compare, for example, the first and last data points of the green and red curves. It is also clear that, for the same execution time, **BDPT** consistently outperforms **PT**, as initially hypothesized. For one sample, **PT** evaluates one forward path (recall section 5.2), whereas **BDPT** assesses a pair of (forward, backward) connections, which corresponds to L paths. Assessing these L paths is quite efficient, resulting in the curves' variance versus time being only slightly right-shifted compared to the variance versus samples curves. The gains in variance range from one to six orders of magnitude, with larger circuits exhibiting larger gains.

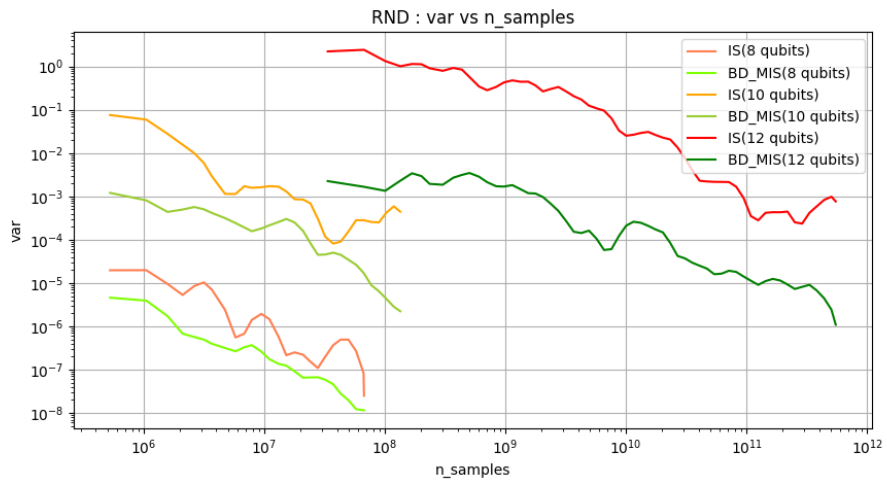
Figure 20 illustrates how efficiency varies with the number of samples. Since efficiency is the reciprocal



(a) SU 5, 15 and 25 qubits

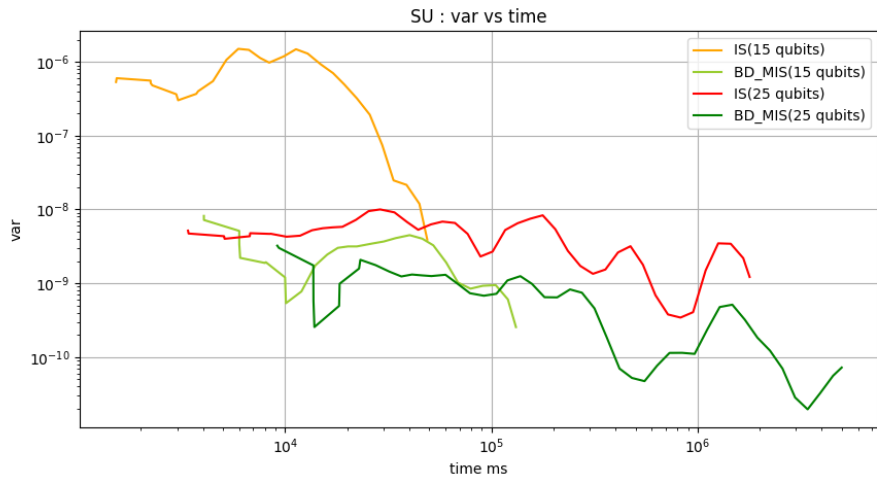


(b) IQP 5, 15 and 25 qubits

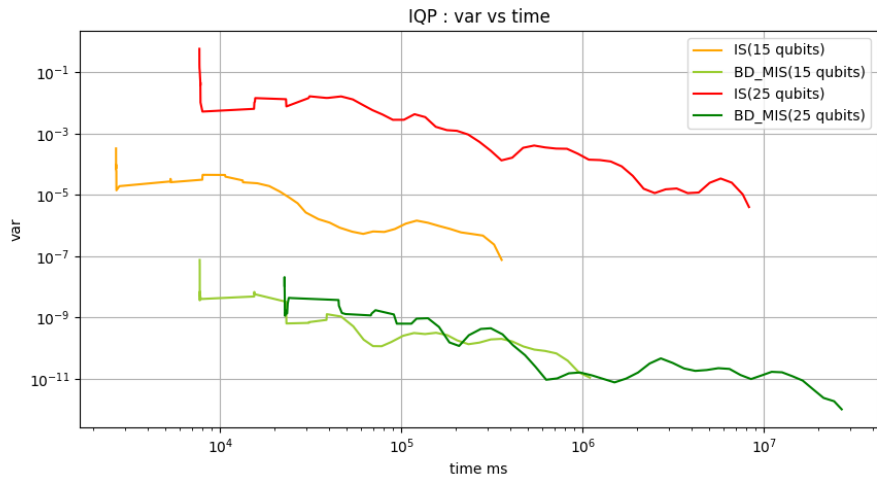


(c) RND 8, 10 and 12 qubits

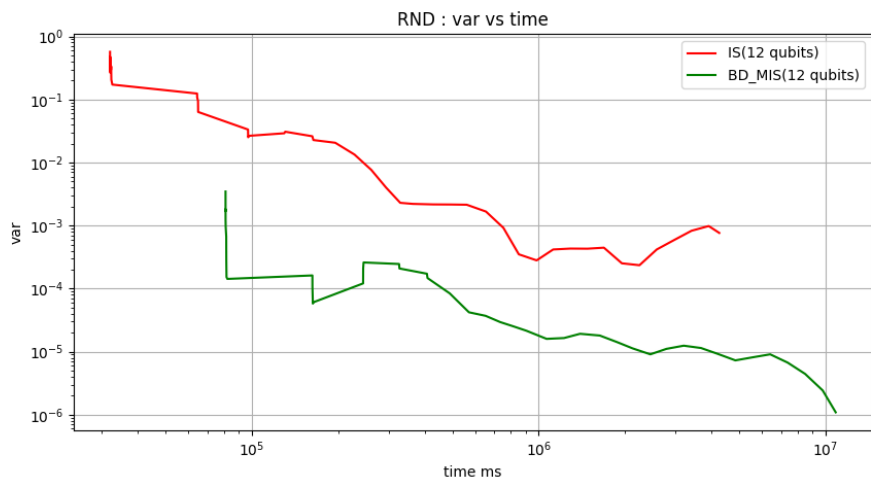
Figure 18: Variance as a function of the number of Monte Carlo samples



(a) SU 5, 15 and 25 qubits



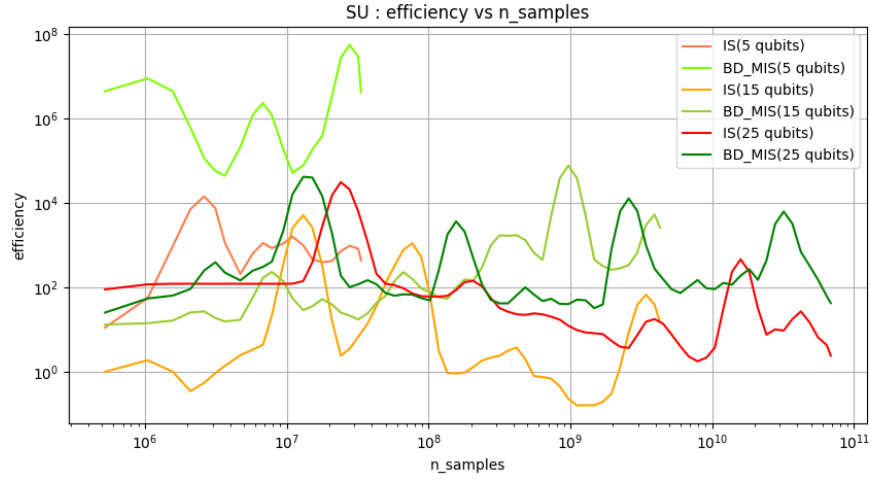
(b) IQP 5, 15 and 25 qubits



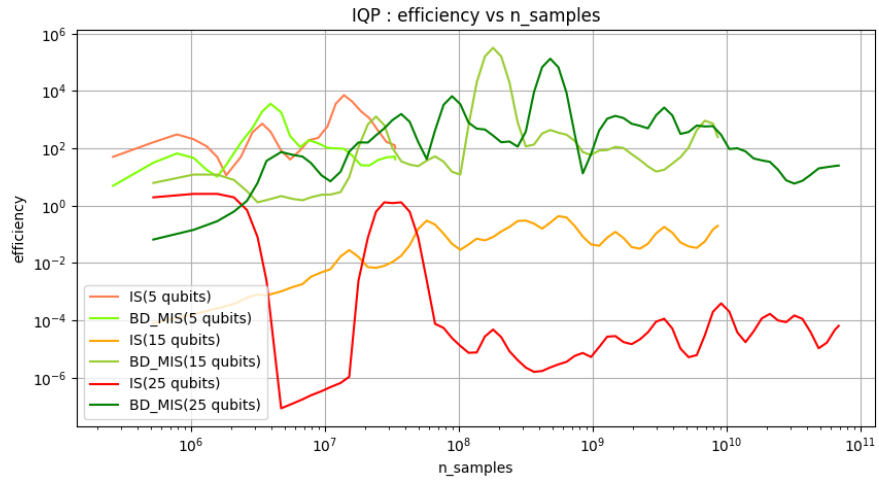
(c) RND 8, 10 and 12 qubits

Figure 19: Variance as a function of time

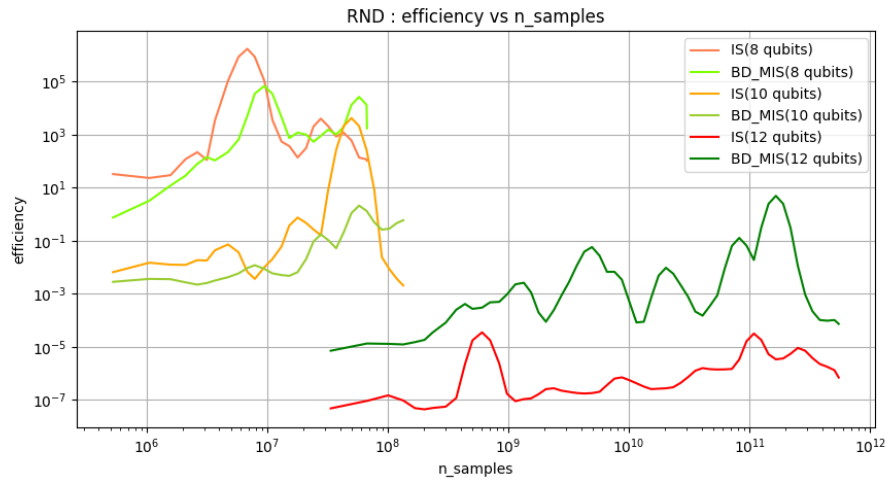
of the product of time by the high-frequency variance, the resulting curve is very peaky, making the curves harder to interpret. The overall trend can still be identified with **BDPT** being more efficient than **PT** , especially for larger circuits. Note, however, that this does not mean that larger circuits, for the same algorithm, are more efficiently simulated than smaller ones; in fact, larger circuits require larger execution times for the same variance, which penalizes the efficiency metric.



(a) SU 5, 15 and 25 qubits



(b) IQP 5, 15 and 25 qubits



(c) RND 8, 10 and 12 qubits

Figure 20: Efficiency as a function of the number of samples

BDPT converges much faster than **PT** because it has a much larger probability of discovering paths with non-zero amplitude, i.e., paths that actually contribute to the amplitude being estimated. This is illustrated in figure 21 for the larger circuits of each family (**BDPT** results are plotted with a solid line, **PT** with a dashed one; the same colour is used for both algorithms for the same circuit). For **SU25** and **IQP25**, and for the same execution time, **BDPT** finds around six orders of magnitude more non-zero paths than **PT**, whereas for **RND12** the gain is about two orders of magnitude. The bidirectional approach, by connecting forward and backward generated paths, is much more successful in building viable paths. Using **MIS** to evaluate the weights ensures good convergence. This agrees with the results exhibited by **Global Illumination** path tracing algorithms.

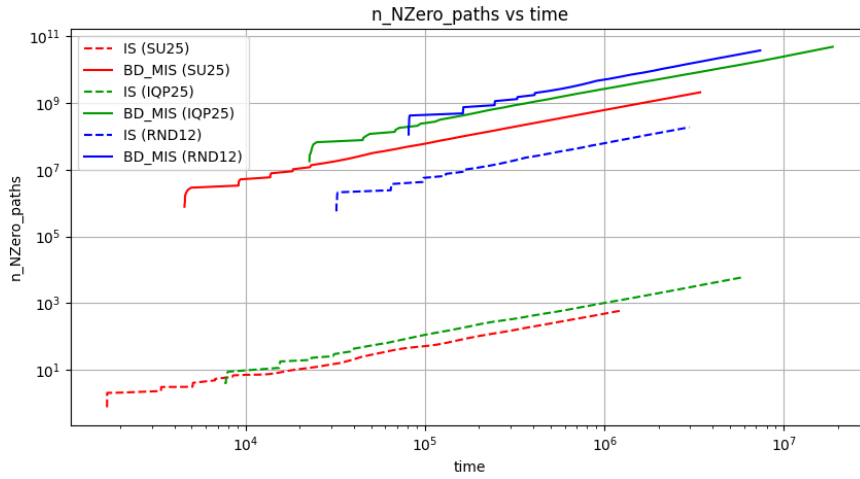


Figure 21: Number of non-zero amplitude paths as a function of time

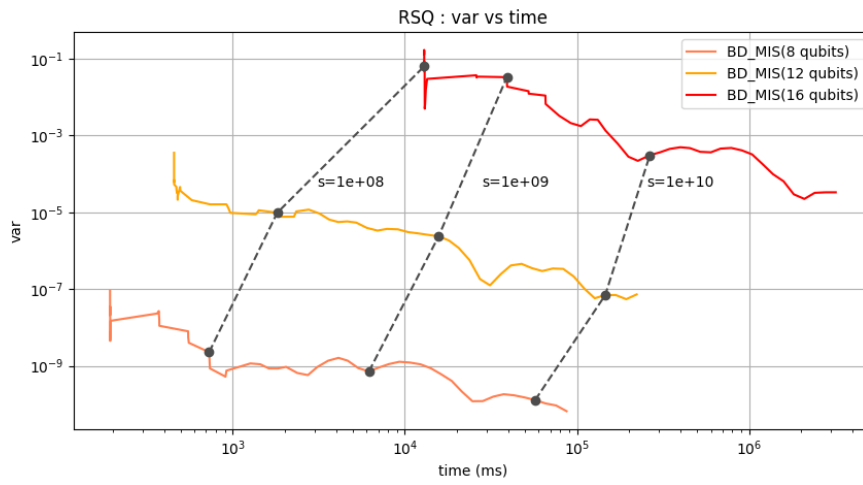
6.2.2 Scalability with Circuit Dimension

Figures 22a and 22b illustrate, for the 8, 12, and 16 qubits **Random Square Circuits (RSQ)** and **BDPT**, how variance changes as a function of execution time and number of samples, respectively. All experiments were run on a single computing node of Deucalion, with 128 computing cores, such that execution times are comparable.

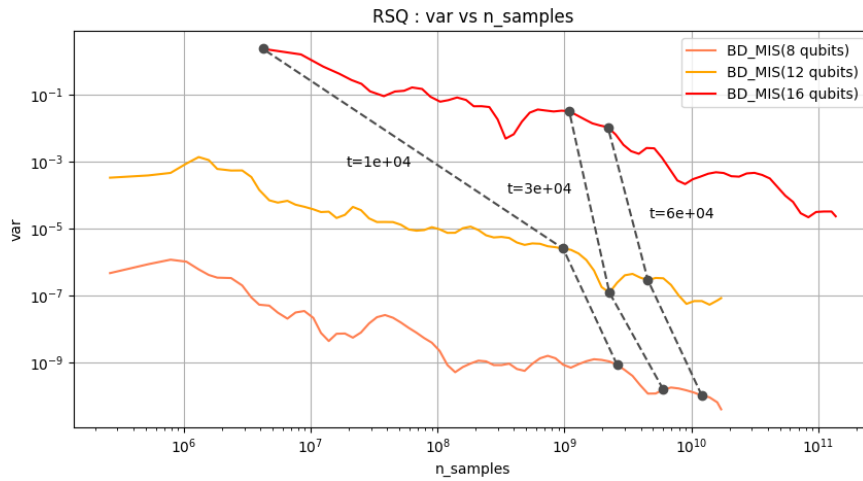
Figure 22a shows that as the circuit dimension increases quadratically, the computation time required to achieve similar variance values increases exponentially by about 3 orders of magnitude. This exponential growth is as expected, since the **RSQ** circuits are not efficiently classically simulable. The black dashed lines are iso number of samples curves. They connect data points obtained for the same number of samples for the three circuits. It is easy to conclude that for the same number of samples variance

degrades approximately 4 orders of magnitude, while still requiring approximately one order of magnitude more computing time.

Figure 22b further corroborates the exponential growth of required classical resources with increasing circuit size. A quadratic increase in circuit size requires three orders of magnitude more samples to achieve the same variance. The black dashed lines are iso time curves. They connect data points obtained for the same execution time for the three circuits. It shows that for the same execution time the variance degrades between 3 to 5 orders of magnitude, with this degradation being larger for the transition from **RSQ12** to **RSQ16**.



(a) Variance versus Time



(b) Variance versus Number of Samples

Figure 22: Scalability with circuit dimension: **RSQ8**, **RSQ12** and **RSQ16**

6.2.3 Very Large Hidden Shift Circuits

The previous sections have demonstrated that the bidirectional approach converges faster than the forward-only algorithm, and it does so by increasing the probability of finding non-zero paths by several orders of magnitude. However, the motivation to test Feynman sum of paths algorithms is their space complexity, which is $\mathcal{O}(nL)$, opening up the possibility of simulating very large circuits.

Table 4 presents the results of simulating **Hidden Shift Circuits (HSC)** with 256, 1024 and 4096 qubits, which would be impossible to process with a naive Schrödinger type simulator with exponential memory requirements. 4, 8, and 16 computing nodes have been used to simulate each of the circuits, respectively.

The data shows that **BDPT** is capable of estimating the desired transition amplitude by finding a reduced number of non-zero paths. The forward only **PT** algorithm did not find any non-zero path for up to 2^{26} samples.

The capability to simulate a 4096-qubit circuit is quite remarkable. A Schrödinger based approach would require memory proportional to $\mathcal{O}(2^{4096})$, which is unfeasible.

Hidden Shift Circuits have been shown to be efficiently simulable using classical resources [27]. It is clear from table 4 that as the circuit width increases 4 fold, the simulation time increases linearly by the same amount. The reason why it doubles instead of quadrupling is that the number of computing nodes also doubled. Classical resources therefore increase linearly, rather than exponentially, with the **HSC** width.

6.3 Discussion

The hypothesis that Bidirectional **Monte Carlo** methods (**BDPT**) are more efficient than forward-only methods (**PT**) in the estimation of quantum circuits transition amplitudes was tested experimentally.

It has been shown that **BDPT** often achieves orders of magnitude less variance than **PT** for the same number of samples (figure 18). Although the former algorithm requires more time than the latter to process the same number of samples, it has also been shown that it converges to smaller variances for the same execution time (figure 19). As hypothesized **BDPT** is more efficient than **PT** in almost all experimentally evaluated scenarios (figure 20).

#qb	depth	nodes	threads	samples	time (h:m:s)	#NzPaths	Var
256	43	4	512	2^{22}	1m43s	113	$1e(-2)$
				2^{23}	3m28s	248	$3e(-3)$
				2^{24}	6m51s	511	$4e(-6)$
				2^{25}	14m05s	993	$9e(-4)$
				2^{26}	31m36s	2018	$2e(-4)$
1024	43	8	1024	2^{22}	3m29s	129	$6e(-5)$
				2^{23}	7m13s	270	$3e(-3)$
				2^{24}	13m56s	502	$5e(-4)$
				2^{25}	28m14s	1003	$4e(-4)$
				2^{26}	55m20s	2025	$1e(-4)$
4096	43	16	2048	2^{22}	7m22s	135	$3e(-3)$
				2^{23}	14m47s	255	$1e(-3)$
				2^{24}	29m31s	501	$4e(-3)$
				2^{25}	1h00m50s	959	$5e(-4)$
				2^{26}	1h57m48s	2070	$1e(-4)$

Table 4: **BDPT** simulation of very large **Hidden Shift Circuits**

Furthermore, experimental results show that **BDPT**’s efficiency is due to its higher probability of finding non-zero paths within the space of paths connecting $|j\rangle$ to $|i\rangle$: $\mathcal{P}_{L,n}^{j,i}$ – see figure 21. For interesting, useful, quantum circuits, this is a very sparse space, meaning that the number of paths with non-zero amplitude (i.e. $\in \overline{\mathcal{Z}}_{L,n}^{j,i}$) is orders of magnitude smaller than the number of paths in $\mathcal{P}_{L,n}^{j,i}$.

Experimental scalability evaluation was performed using random square circuits, which are believed to be hard to simulate classically. Results corroborate that exponentially growing classical resources are still required to simulate quadratically growing circuits – figure 22. This is as expected: if linearly growing quantum systems can be simulated with less than exponentially growing classical resources, then there is no quantum computational advantage.

Finally, it has been demonstrated that the proposed **BDPT** algorithm can simulate circuits with up to 4096 qubits without encountering any memory barrier. These are **Hidden Shift Circuits (HSC)**, whose deterministic output state is known by design, which allows verification of the **Monte Carlo** simulation.

Chapter 7

Concluding Remarks

Stupidity lies in wanting to draw conclusions.

Letter to Louis Bouilhet, September 1850

Gustave Flaubert

The art and science of asking questions is the source of all knowledge.

1924, Thomas Berger

This lecture proposes the application of a bidirectional **Monte Carlo** algorithm, inspired by the best practices in the area of **Global Illumination**, to the problem of simulating quantum circuits using classical resources. In detail, the problem of estimating the transition amplitude $\langle i | \hat{U} | j \rangle$, where $|i\rangle$ and $|j\rangle$ are basis states in the computational basis and $\hat{U}$ is the time evolution operator describing the circuit of interest.

The proposed algorithm stochastically generates, for each sample, a forward path segment (starting in the initial state) and a backward path segment (starting in the final state). After each layer of gates, the forward and backward segments are deterministically connected, establishing a full path from $|j\rangle$ to $|i\rangle$. The contributions of many such paths are carefully weighted using the **Monte Carlo** quadrature and **Multiple Importance Sampling**. This algorithm is referred to as BiDirectional Path Tracing (**BDPT**).

BDPT is evaluated against a forward-only algorithm, which generates a path starting in $|j\rangle$ and stochastically progressing forward in time across the circuit's layers. The last connection to the final state $|i\rangle$ is deterministic and, in the vast majority of cases, will have amplitude 0, rendering the whole path contribu-

tion null. This algorithm is referred to as Forward Path Tracing (**PT**).

Experimental results have shown that, for the diversity of circuits included in the experiments, **BDPT** converges to the true value of the transition amplitude much faster (both in time and number of samples) than **PT**. And it does so by finding more non-zero paths for the same computational effort. **BDPT** does indeed increase the probability of finding paths in $\overline{\mathcal{Z}}_{L,n}^{j,i}$ by several orders of magnitude compared to **PT**. And this advantage increases with the circuit size.

Since Feynman's sum of paths approach to classical quantum simulation is not exponential in memory requirements with the circuit size, it enables the simulation of circuits with a very high count of qubits. This lecture demonstrated deterministic **Hidden Shift Circuits** with up to 4096 qubits, which is a remarkable achievement.

The path integral simulation of quantum processes allows for the elimination of exponential memory requirements. However, it dramatically increases the time requirements, which can be a very challenging obstacle to its adoption. **Monte Carlo** simulations, by computing stochastic approximation rather than exact solutions, help in curbing these time requirements. Approximate solutions, however, might not be acceptable for all applications. And really hard quantum systems are known to require an exponential number of samples due to the so-called numerical sign problem [28, 29].

A versatile, time and memory-efficient simulator will most likely integrate features from path integration and Monte Carlo simulation with other well-established techniques, such as circuit knitting [17] and tensor decomposition [16], for example.

Future Work

The work just described raises a plethora of new research questions, of which four are singled out as directions for future work.

Numerical sign problem

The proposed bidirectional algorithm improves on the forward-only algorithm due to its ability to find non-zero paths with much higher probability. Bidirectional algorithms handle the sparseness of the integration

domain.

However, another more challenging problem plagues a large number of quantum **Monte Carlo** simulations. Since the paths' amplitudes are complex values with possibly positive or negative real and imaginary parts, paths can, and often do, cancel with each other when added, in a process referred to as destructive interference. Since Monte Carlo algorithms sample the path space stochastically, an exponential number of samples is often required to find cancelling groups of paths. Otherwise, the estimator variance will be large. This is the *numerical sign problem* [28, 29]. No universal solution is known, although solutions are known for different classes of problems.

As an example, the transition amplitude $\langle 1 | \hat{U}_{\text{RND8}} | 0 \rangle$ (circuit **RND8**, see table 2) is $0 + 0j$, but out of the 2^{64} paths connecting $|0\rangle$ to $|1\rangle$, 512 have non-zero amplitude. However, they perfectly cancel each other out. With circuit **RND12** $\langle 0 | \hat{U}_{\text{RND12}} | 0 \rangle = 0 + 0j$, but there are $\approx 10^9$ non-zero paths that cancel each other (out of $\approx 10^{61}$ paths).

The relevant research questions are:

- Is it possible to know beforehand whether the numerical sign problem will manifest itself for a given circuit?
- can a **MC** algorithm be developed, which avoids or mitigates the numerical sign problem?

Simulation of open quantum systems

This lecture considered only ideal, noise-free, closed quantum systems. However, real quantum devices are subject to a diversity of noise sources that dramatically impact the final result. Accurate simulations of quantum circuits must model them as open quantum systems and include realistic noise models. The action of noisy quantum channels on the quantum state can be described using Kraus operators [30]. Kraus operators model different sources of noise as stochastic processes. For example, under the action of an amplitude damping channel, the ground state ($|0\rangle$) of a qubit remains unaffected, whereas the excited state ($|1\rangle$) decays to the ground state with a given probability. Integrating such models of noisy quantum channels into **MC** estimators should be simple and effective, since they share a stochastic approach to problem solving.

There have been proposals to leverage open quantum systems' noise in the context of deterministic clas-

sical simulation and (Pauli) sum of paths. It has been shown that circuits believed to be exponentially hard to simulate, might not be so under realistic conditions [31]. Extending these results into the **MC** framework is a most promising line of research.

The relevant research questions are:

- can stochastic models of noise be seamlessly integrated into the proposed **MC** Feynman sum of paths algorithms, maintaining the favourable space complexity?
- can **BDPT** be applied in the noisy context, given that some processes (e.g., amplitude damping) are non-reversible?

Machine learning

The **MC** algorithms discussed in this lecture will suffer from the numerical sign problem if the paths' weights are complex and cancel each other out. Some authors have proposed, for some types of problems, using machine learning approaches, combined with **MC** sampling, to circumvent or mitigate the sign problem [32, 33]. Neural networks are trained and can then provide predictions for the quantity being estimated.

The relevant research question is:

- can machine learning techniques be used, together with the proposed algorithms, to mitigate the numerical sign problem?

Bibliography

- [1] R. P. Feynman, “Space-time approach to non-relativistic quantum mechanics,” *Rev. Mod. Phys.*, vol. 20, pp. 367–387, Apr 1948. [Online]. Available: <https://link.aps.org/doi/10.1103/RevModPhys.20.367>
- [2] B. Rudiak-Gould, “The sum-over-histories formulation of quantum computing,” 2006. [Online]. Available: <https://arxiv.org/abs/quant-ph/0607151>
- [3] E. Lafortune and Y. Willems, “Bi-directional path tracing,” *Proceedings of Third International Conference on Computational Graphics and Visualization Techniques (Compugraphics’)*, vol. 93, 01 1998.
- [4] E. Veach, “Robust Monte Carlo Methods for Light Transport Simulation,” Ph.D. dissertation, Stanford University, 1997.
- [5] B. M. Austin, D. Y. Zubarev, and W. A. J. Lester, “Quantum monte carlo and related approaches,” *Chemical Reviews*, vol. 112, no. 1, pp. 263–288, 2012.
- [6] R. Feynman and A. Hibbs, *Quantum Mechanics and Path Integrals*, 1st ed. McGraw-Hill, 1965.
- [7] M. Kalos and P. Whitlock, *Monte Carlo Methods*, 2nd ed. Wiley-VCH, 2008.
- [8] J. T. Kajiya, “The rendering equation,” in *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '86. Association for Computing Machinery, 1986, p. 143–150.
- [9] M. Hanada and S. Matsuura, *MCMC from Scratch: A Practical Introduction to Markov Chain Monte Carlo*, 1st ed. Springer, 2022.
- [10] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, “Equation of state calculations by fast computing machines,” *J. Chem. Phys.*, vol. 21, no. 6, p. 1087–1092, 1953. [Online]. Available: <https://doi.org/10.1063/1.1699114>

- [11] J. W. S. Rayleigh, "On the law of reciprocity in diffuse reflexion," *Philosophical Magazine*, vol. 49, p. 324–325, 1900.
- [12] W. R. Hamilton, *On a General Method of Expressing the Paths of Light, & of the Planets, by the Coefficients of a Characteristic Function*. PD Hardy, 1833.
- [13] K. Young, M. Scese, and A. Ebneenasir, "Simulating quantum computations on classical machines: A survey," 2023. [Online]. Available: <https://arxiv.org/abs/2311.16505>
- [14] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.
- [15] A. Zulehner and R. Wille, "Advanced simulation of quantum computations," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 5, p. 848–859, 2018.
- [16] I. Markov and Y. Shi, "Simulating quantum computation by contracting tensor networks," *SIAM Journal on Computing*, vol. 38, no. 3, p. 963–981, 2008.
- [17] S. Yang and P. Murali, "Understanding the scalability of circuit cutting techniques for practical quantum applications," 2024. [Online]. Available: <https://arxiv.org/abs/2411.17756>
- [18] W. Heisenberg, "Über quantentheoretische umdeutung kinematischer und mechanischer beziehungen," *Zeitschrift für Physik*, vol. 33, no. 1, pp. 879–893, 1925, english translation: *On the Quantum-Theoretical Reinterpretation of Kinematic and Mechanical Relations*.
- [19] C. Gini, "On the Measure of Concentration with Special Reference to Income and Statistics," *Colorado College Publication*, vol. 208, pp. 73–79, 1936.
- [20] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, p. 242–246, 2017.
- [21] G. Aleksandrowicz, T. Alexander, P. Barkoutsos, L. Bello, Y. Ben-Haim, D. Bucher, A. Carballo-Franquis, C.-F. Chen, A. Cross, J. Cruz-Benito, and et al., "Qiskit: An open-source framework for quantum computing," 2019, accessed: 2025-07-28. [Online]. Available: <https://qiskit.org>
- [22] M. J. Bremner, R. Jozsa, and D. J. Shepherd, "Classical simulation of commuting quantum computations implies collapse of the polynomial hierarchy," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 467, no. 2126, pp. 459–472, 2011.

- [23] S. C. Marshall, S. Aaronson, and V. Dunjko, “Improved separation between quantum and classical computers for sampling and functional tasks,” 2024, accessed: 2025-07-28. [Online]. Available: <https://doi.org/10.48550/arXiv.2410.20935>
- [24] S. Boixo, S. Isakov, and V. Smelyanskiy, “Characterizing quantum supremacy in near-term devices,” *Nature Physics*, vol. 14, pp. 595–600, 2018.
- [25] F. Arute, K. Arya, and R. Babbush, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, pp. 505–510, 2019.
- [26] F. C. R. Peres and E. F. Galvão, “Quantum circuit compilation and hybrid computation using Pauli-based computation,” *Quantum*, vol. 7, p. 1126, Oct. 2023. [Online]. Available: <https://doi.org/10.22331/q-2023-10-03-1126>
- [27] M. Amy and L. S. Stinchcombe, “Polynomial-time classical simulation of hidden shift circuits via confluent rewriting of symbolic sums,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.02778>
- [28] E. Y. Loh, J. E. Gubernatis, R. T. Scalettar, S. R. White, D. J. Scalapino, and R. L. Sugar, “Sign problem in the numerical simulation of many-electron systems,” *Physical Review B*, vol. 41, no. 13, pp. 9301–9307, 1990.
- [29] G.-R. Pan and Z. Y. Meng, “Sign problem in quantum monte carlo simulation,” 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2204.08777>
- [30] D. A. Lidar, “Lecture notes on the theory of open quantum systems,” 2020. [Online]. Available: <https://arxiv.org/abs/1902.00967>
- [31] D. Aharonov, X. Gao, Z. Landau, Y. Liu, and U. Vazirani, “A polynomial-time classical algorithm for noisy random circuit sampling,” in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*. ACM, Jun. 2023, p. 945–957.
- [32] P. Broecker, J. Carrasquilla, R. Melko, and S. Trebst, “Machine learning quantum phases of matter beyond the fermion sign problem,” *Scientific Reports*, vol. 8823, no. 7, 2017.
- [33] J.-L. Wynen, E. Berkowitz, S. Krieg, T. Luu, and J. Ostmeyer, “Machine learning to alleviate hubbard-model sign problems,” *Phys. Rev. B*, vol. 103, p. 125153, Mar 2021.

Appendix A

Circuit Layer Level Methods

This appendix describes routines used throughout the document's algorithms to evaluate quantities of interest across a quantum circuit's layer.

The actual implementations depend on how the circuit, its layers, gates, and qubits are represented. Descriptions are therefore made at an intermediate level, combining pseudo-code with mathematical expressions, to remain largely independent of implementation details.

A.1 Preliminaries

A quantum circuit's unitary time evolution operator $\hat{U}$ can be decomposed into a sequence of simpler unitaries $\hat{U}_l$, corresponding to discretizing time into L contiguous subintervals. These unitaries compose via matrix multiplication, $\hat{U} = \prod_{l=L-1}^0 \hat{U}_l$, being commonly referred to as layers of the quantum circuit – see section 4.2.

The subscript $_{(j)}$ is used to index bits (or qubits) within a sequence, with $_{(j)}$ being a tuple with the relevant indices. If i is an integer, then $i_{u,v}$ indexes bits with order u and v . Let $i = 144_{b10} = 10010000_{b2}$. Then $i_{(4,1)} = 10_{b2} = 3_{b10}$.

A layer, in the context of this lecture, has a maximum depth of one gate, with each gate operating on a subset of 1 or 2 qubits. The gates' unitaries, $\hat{G}$, compose through tensor products, with $\hat{U}_l = \bigotimes_j \hat{G}_{l,(j)}$, where j is a tuple identifying the qubits the gate acts upon. Since each layer has a maximum depth of one, within a layer each qubit is acted upon, at most, by a single gate.

A single qubit gate is described by a 2×2 matrix and a 2-qubits gate by a 4×4 matrix. The gate's transition amplitude from input basis state $|i\rangle$ to output basis state $|o\rangle$ is given by the matrix element $\hat{G}[o, i]$.

Consider, for example, the Hadamard gate matrix:

$$\hat{H} = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}$$

The amplitude $\langle 0|\hat{G}|1\rangle$ is given by $\hat{G}[0, 1] = 1/\sqrt{2}$ and $\langle 1|\hat{G}|1\rangle = \hat{G}[1, 1] = -1/\sqrt{2}$. The input basis state can be viewed as a column selector, while the output basis state serves as a row selector.

The column i of the unitary, $\hat{G}[:, i]$, can be used to compute the probability distribution over all possible outputs for the input basis state $|i\rangle$. Such a probability distribution is given by the vector formed by the square amplitude of each element of $\hat{G}[:, i]$. For a Hadamard gate the probability distribution over the output given input state $|1\rangle$, $p(|o\rangle | |1\rangle)$, is

$$\hat{H}[:, 1] = \begin{Bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{Bmatrix} \Rightarrow p(|o\rangle | |1\rangle) = \begin{Bmatrix} 1/2 \\ 1/2 \end{Bmatrix}$$

A.2 Layer Amplitude Evaluation

```
amplitude ← EvalLayer (CircuitLayers, l, c_state, n_state)
```

Computes the transition amplitude from the current state (`c_state`) to the next state (`n_state`) across circuit's layer 1:

$$\langle \mathbf{n_state} | \hat{U}_l | \mathbf{c_state} \rangle = \prod_j \hat{G}_{l,(j)}[\mathbf{n_state}_{(j)}, \mathbf{c_state}_{(j)}]$$

where j iterates over qubits and pairs of qubits (for 2-qubit gates) and $\hat{G}_{l,(j)}$ is the gate operating on qubit tuple (j) .

A.3 Layer Sampling

```
n_state, pNextState ← SampleLayer (CircuitLayers, l, c_state)
```

Given the input state, `c_state`, stochastically samples an output state from the probability distribution defined by the quantum circuit layer. Returns the sampled next state and the probability with which it was sampled.

Sampling is performed per gate, with the output state being built by (re)setting the qubits operated by each gate, and the final probability being the product of each gate's sampled output.

Algorithm 6: Sample Layer

Data: CircuitLayers, l, c_state

Result: n_state, pNextState

```

1 pNextState  $\leftarrow$  1. ;
2 foreach  $\hat{G}_{l,(j)}$  in layer l do                                // iterate all gates
3   | n_state(j), thisProb  $\leftarrow$  SampleFrom( $\hat{G}_{l,(j)}$ [:,c_state(j)]) ;
4   | pNextState  $\leftarrow$  pNextState * thisProb ;
5 end
6 return n_state, pNextState ;

```

A.4 Layer Transition Probability

thisConnectProb $\leftarrow$ EvalLayerProb (CircuitLayers, l, c_state, n_state)

The probability of evolving from classical input state c_state to classical output state n_state across layer $\hat{U}_l$ is given by the product of the respective squared amplitudes at the gate level:

$$p(\mathbf{n_state} \mid \mathbf{c_state}, \hat{U}_l) = \prod_j \left| \hat{G}_{l,(j)}[\mathbf{n_state}_{(j)}, \mathbf{c_state}_{(j)}] \right|^2$$

